

Final Project: Time-Series Data and Application to Stock Markets

1. Introduction:

Stock market forecasting is a challenging problem because prices are non-stationary, noisy, and influenced by factors that span human behavior, macroeconomics, and market microstructure. This project approaches the problem empirically: rather than assuming any single model or feature set is optimal, we systematically evaluate design choices — data filtering, feature engineering, model architecture, training protocol, and evaluation strategy — at each stage.

Two datasets were used: historical daily OHLCV (Open, High, Low, Close, Volume) data for Nasdaq-listed stocks, and the same for Vietnamese stocks listed on HOSE, HNX, and the UpCom exchange. The project is organized into four main analytical tasks: price prediction (Tasks 1 and 2), trading signal classification (Task 3), and portfolio construction with risk management (Task 4).

2. Shared Methodology:

2.1. The Time-Series Constraints

A core methodological principle throughout the project is that **temporal ordering must never be violated**. All data splits are chronological: the earliest 70% of records form the training set, the next 15% form the validation set, and the final 15% form the test set. No shuffling is applied at any stage. This ensures the model is always evaluated on data that is strictly "in the future" relative to its training window, which is the only valid way to assess a stock forecasting system.

Cross-validation uses `TimeSeriesSplit` from scikit-learn, which implements an expanding-window scheme. Each fold adds more training data while the validation window shifts forward in time. This is in contrast to random k-fold CV, which would contaminate future information in training folds and produce falsely optimistic results.

2.2. Company Selection and Data Quality Filtering

Not all listed companies are suitable for modeling. The filtering pipeline applies the following criteria to every ticker before it enters any experiment:

- **Minimum history:** At least 120 trading days of data (approximately six months), ensuring enough sequence history to train a 30-day lookback model.
- **Spike rate:** The daily return exceeds $\pm 8\%$ on fewer than 8% of trading days, excluding tickers with extreme price discontinuities likely caused by data errors or corporate events.
- **Volume activity:** The volume is non-zero on at least 90% of trading days, excluding illiquid stocks where price discovery is unreliable.

- **Flat price rate:** Fewer than 15% of consecutive days have identical closing prices, which would indicate stale or suspended quotes.
- **Minimum median volume:** At least 10,000 shares traded per day.

Each eligible ticker is also assigned a composite stability score that penalizes volatility, spike frequency, and flat-price periods, and rewards liquidity. For single-ticker experiments, the ticker with the longest history (Nasdaq) or the best stability score (Vietnam) is selected.

2.3. Feature Engineering

Beyond raw OHLCV prices, each model receives a set of derived technical features:

- **Log Return:** $\log(\text{Close}_t / \text{Close}_{t-1})$ — captures daily proportional price change on a symmetric, approximately normally distributed scale.
- **MA10, MA20:** 10- and 20-day moving averages of the target price — smooth short- and medium-term trends.
- **Volatility10:** Rolling standard deviation of log returns over 10 days — represents recent risk or market uncertainty.

These features provide the model with multi-scale temporal context: raw prices encode level, returns encode momentum, moving averages encode trend, and volatility encodes regime.

2.4. Model Architecture

The primary model is a two-layer Gated Recurrent Unit (GRU) network. The choice of GRU over LSTM was deliberate:

- GRUs have fewer parameters (no separate cell state), making them faster to train and less prone to overfitting on moderate-sized financial datasets.
- In preliminary experiments, LSTM with high recurrent dropout produced a training loss substantially higher than validation loss — a signature of over-regularization, where dropout is active during training but disabled at inference. GRU with lower dropout resolved this.

The architecture is: `Input → GRU(48 units) → Dropout → GRU(24 units) → Dropout → Dense(output_dim)`.

Huber loss is used instead of MSE. Huber loss behaves like MSE for small errors and like MAE for large errors, making it robust to occasional extreme price movements that would otherwise dominate gradient updates with MSE.

Adam optimizer with a learning rate of 3×10^{-4} is used throughout. `ReduceLROnPlateau` halves the learning rate when validation loss stagnates, and `EarlyStopping` with patience of 8 epochs restores the best weights when overfitting begins.

2.5. Evaluation Framework

For regression tasks (Tasks 1 and 2), the following metrics are reported:

- **MAE / RMSE:** Absolute prediction error in original price units. Lower is better, but these numbers are scale-dependent and should not be compared across tickers with different price magnitudes.
- **sMAPE (Symmetric MAPE):** Scale-independent percentage error. An SMAPE below 5% is strong for daily price prediction; above 20% indicates the model is barely better than a naive guess.

- **R²:** Proportion of price variance explained. Note: R² near 0.98 is common and does not indicate a useful model — a naive "predict today's price for tomorrow" model also achieves $R^2 \approx 0.98$ on any trending stock.
- **Directional Accuracy (DA):** The percentage of test periods where the model correctly predicts whether the price moves up or down. This is the most financially meaningful metric because profitable trading strategies depend on direction, not absolute price. A DA above 55% is commercially useful; near 50% is equivalent to a coin flip.

For this reason, each experiment includes a **naive baseline comparison** that measures the persistent forecaster (predict tomorrow = today) on the same test set. The GRU is only meaningfully better than chance if it exceeds the baseline on DA and sMAPE.

3. Task 1 - Nasdaq Stock Price Prediction

3.1. Setup

The Nasdaq dataset consists of historical daily OHLCV files, one per ticker. After applying the quality filter, the ticker with the longest history was selected for all three subtasks. The full feature set used is: Open, High, Low, Close, Adjusted Close, Volume, Log Return, MA10, MA20, Volatility10 (10 features total).

The data split is 70%/15%/15% chronologically. The lookback window is 30 trading days (approximately six weeks of market data), meaning the model sees the past 30 days to predict one or more future values.

3.2. Task 1.1 — Multi-Feature Next-Day Forecast

This is the baseline experiment. The model predicts the Adjusted Close price one day ahead using all 10 features. `forecast_horizon=1`, `multi_output=False`.

Key insight: Using multiple features (OHLCV + derived) instead of only the Open price provides the model with richer context. Moving averages capture whether the current price is above or below recent trends. Log return normalizes for price scale differences across tickers. Volatility10 signals whether the market is in a calm or turbulent regime — the model can learn to make more conservative predictions when volatility is elevated.

Baseline comparison observation: The persistent model (predict tomorrow = today) achieves a high R² because stock prices are autocorrelated — tomorrow's price is usually close to today's. The meaningful question is whether the GRU's directional accuracy exceeds the persistent baseline. If it does, the model has learned something about market dynamics beyond simple persistence.

3.2.1 Task 1.1 - Results

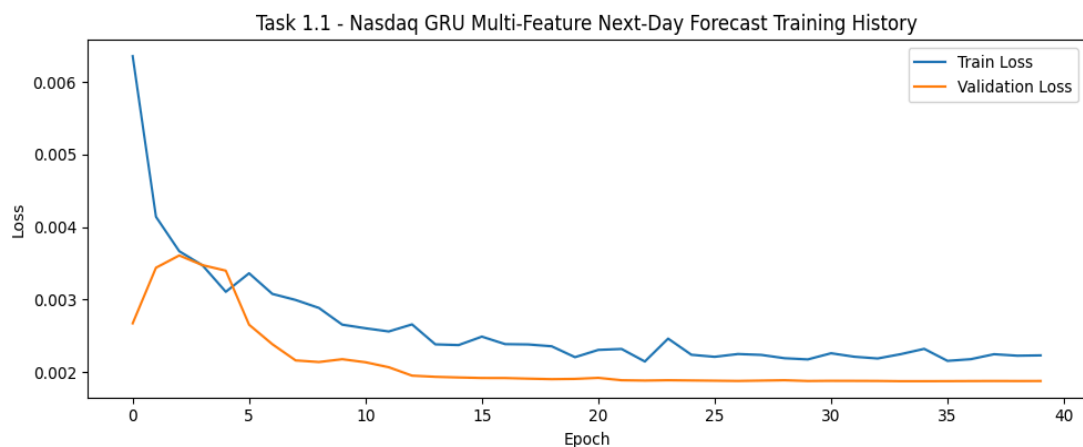


Figure 3.1: Task 1.1 GRU Training and Validation Loss

The training and validation losses both decrease and stabilize, showing that the GRU converges properly. Validation loss stays close to or slightly below training loss, suggesting the model is not severely overfitting. The stable validation curve supports using the trained model for test evaluation.

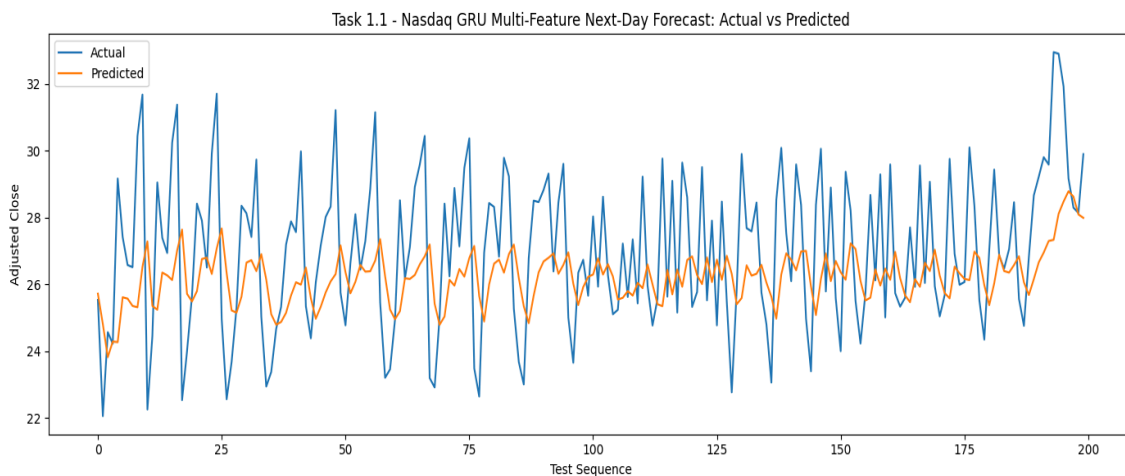


Figure 3.2: Task 1.1 Actual vs Predicted Adjusted Close Price

The predicted curve **follows the general price level and trend**, but is **smoother than the actual curve**. The model **captures broad movement** but **does not reproduce short-term daily spikes**. This is **expected for GRU regression** because **the model learns average temporal patterns rather than exact market noise**.

```

Task 1.1 - Nasdaq GRU Multi-Feature Next-Day Forecast Metrics
MAE: 2.3361
MSE: 9.2088
RMSE: 3.0346
MAPE: 13.55%
sMAPE: 13.10%
R2: 0.8358
Directional Accuracy: 42.31%

```

Table 3.3: Task 1.1 Nasdaq Next-Day Forecast Metrics

Task 1.1 achieves $R^2 = 0.8358$ and $MAPE = 13.55\%$, indicating **strong price-level prediction performance**. $RMSE = 3.03$ means the typical prediction error is around 3 adjusted-close price units. **Directional accuracy is 42.31%**, so the model is **better at estimating price level than predicting daily movement direction**.

3.3. Task 1.2 — k-th Day Forecast (k=3)

The model is retrained with `forecast_horizon=3, multi_output=False`. This means the input window of 30 days is used to predict the Adjusted Close **exactly 3 days ahead**, skipping the intervening days.

New concept — forecasting horizon degradation: Prediction error grows with forecasting horizon. This is expected and documented in the financial forecasting literature. A model predicting 3 days ahead faces more uncertainty than one predicting 1 day ahead, because 3 days of unpredictable noise accumulate. The rubric explicitly states no fixed accuracy threshold is required; the methodology and its justification are what matter.

Architecture decision: `multi_output=False` is correct here because we want only the k-th day value — not the full sequence between now and k. The model learns to project the GRU's hidden state directly to the distant future target.

3.3.1 Results

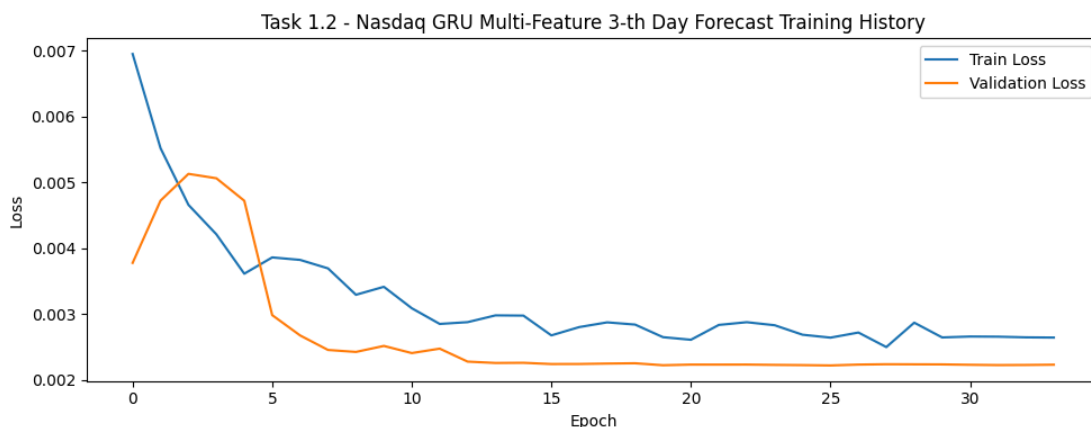


Figure 3.2.1: Task 1.2 — Multi-Feature 3rd-Day Forecast

The training loss decreases steadily and stabilizes after around 15 epochs. The validation loss also drops and remains stable, showing that the GRU converges without severe overfitting. Validation loss being slightly lower than training loss is acceptable because dropout is active during training but disabled during validation.

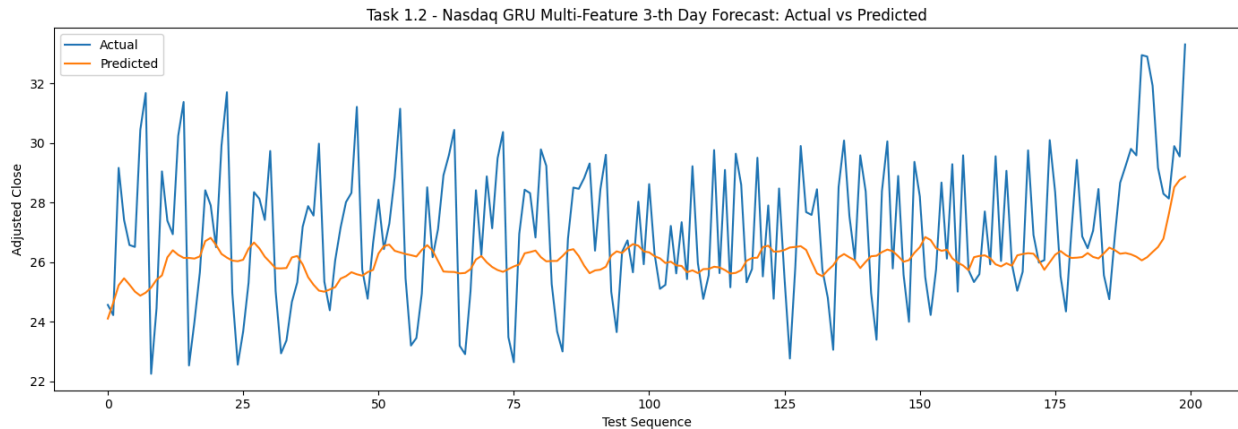


Figure 3.4: Task 1.2 Actual vs Predicted Adjusted Close Price

The predicted values follow the general price level but are smoother than the actual prices. The model does not capture sharp daily fluctuations, which is expected because the 3rd-day forecast is harder than the next-day prediction. However, the prediction remains in the correct price range and follows the broader movement.

Task 1.2 - Nasdaq GRU Multi-Feature 3-th Day Forecast Metrics	
MAE:	2.4790
MSE:	10.3037
RMSE:	3.2099
MAPE:	14.41%
sMAPE:	13.90%
R2:	0.8166
Directional Accuracy:	50.19%

Table 3.4: Task 1.2 Nasdaq 3rd-Day Forecast Metrics

Task 1.2 achieves MAE = 2.4790, RMSE = 3.2099, MAPE = 14.41%, and $R^2 = 0.8166$. This indicates that the model still performs well for the 3rd-day forecast, although the error is slightly higher than Task 1.1 because forecasting further into the future increases uncertainty. Directional accuracy improves to 50.19%, meaning the model is approximately balanced in predicting upward and downward movement.

3.4. Task 1.3 — k Consecutive Days Forecast (k=3)

The model predicts all 3 days ahead simultaneously: day 1, day 2, and day 3. `forecast_horizon=3`, `multi_output=True`. The output layer produces 3 values from a single forward pass.

New concept — **multi-output regression**: Rather than training three separate models, the **multi-output approach shares the GRU encoder across all forecast horizons**. This is more parameter-efficient and forces the model to learn a representation that generalizes across horizons. The trade-off is that earlier forecast steps (day 1) effectively carry a larger gradient signal than later ones (day 3), since day-1 errors are smaller in magnitude.

Observation: The multi-output plot shows the typical pattern — day 1 predictions are closest to actual values, and accuracy degrades progressively for days 2 and 3. This matches the general principle that stock price predictability decays rapidly with horizon.

3.4.1. Results

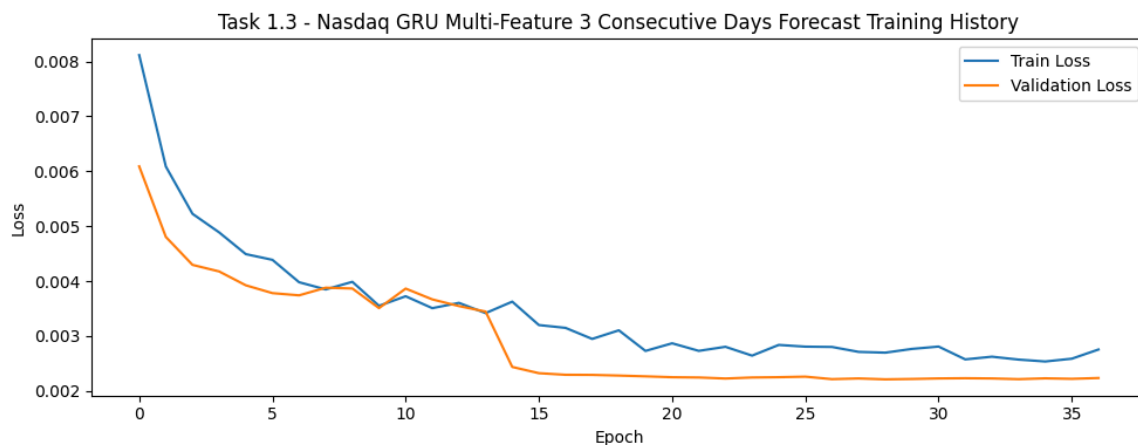


Figure 3.5: Task 1.3 GRU Training and Validation Loss

The training and validation losses decrease and stabilize, showing proper convergence. The validation loss stays close to the training loss, so there is no major overfitting. This supports the model's ability to learn a stable multi-output forecasting pattern.

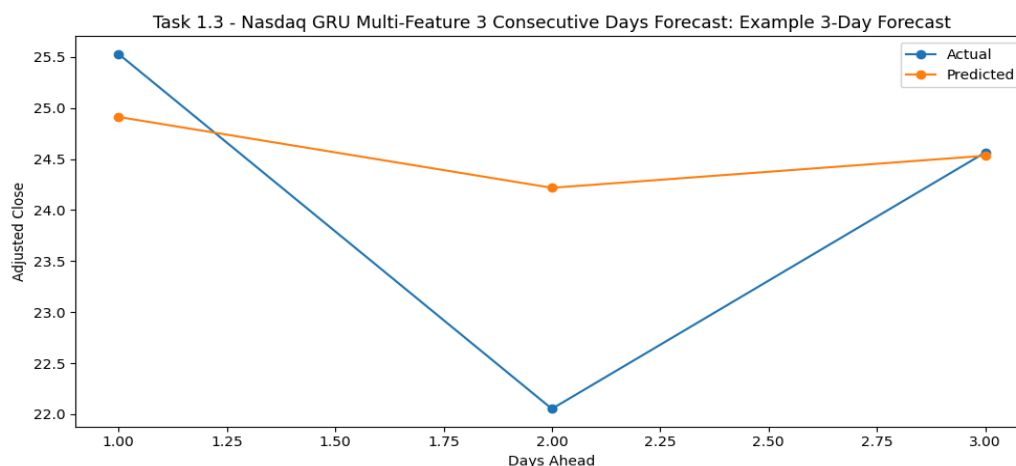


Figure 3.6: Task 1.3 Example 3-Day Forecast

The example forecast shows the model predicting the next three adjusted-close values as a sequence. The predicted curve is smoother than the actual curve and misses the sharp drop on day 2, but it recovers by day 3. This shows that the model captures the general price level but struggles with sudden short-term fluctuations.

```
Task 1.3 - Nasdaq GRU Multi-Feature 3 Consecutive Days Forecast Metrics
MAE: 2.4779
MSE: 10.1788
RMSE: 3.1904
MAPE: 14.26%
sMAPE: 13.84%
R2: 0.8185
Directional Accuracy: 55.11%
```

Table 3.5: Task 1.3 Nasdaq 3-Consecutive-Day Forecast Metrics

Task 1.3 achieves **MAE = 2.4779**, **RMSE = 3.1904**, **MAPE = 14.26%**, and **R² = 0.8185**. These results are close to Task 1.2, showing that the GRU can predict a short sequence of future prices with similar accuracy. Directional accuracy improves to 55.11%, suggesting that the multi-output setup captures short-term movement direction better than the next-day model.

	Task	Model	Description	MAE	MSE	RMSE	MAPE	sMAPE	R2	Directional Accuracy
0	Task 1.1	GRU	Multi-feature next-day forecast	2.336081	9.208758	3.034594	13.547285	13.102747	0.835789	42.312579
1	Task 1.2	GRU	Multi-feature 3-th day forecast	2.479000	10.303674	3.209934	14.406667	13.899241	0.816622	50.191083
2	Task 1.3	GRU	Multi-feature 3 consecutive days forecast	2.477878	10.178839	3.190429	14.258997	13.839722	0.818482	55.112431

Table 3.6: Overall Task 1 Nasdaq Forecasting Performance Summary.

Table 3.6 compares the three Nasdaq forecasting experiments. All three models achieve strong regression performance, with R² above 0.81 and MAPE around 13–14%. Task 1.1 has the best price-level accuracy, with the lowest MAE, RMSE, and MAPE, showing that next-day forecasting is the easiest setting. Task 1.2 and Task 1.3 have slightly higher errors because forecasting further ahead introduces more uncertainty. However, Task 1.3 achieves the best directional accuracy at 55.11%, suggesting that predicting a short 3-day sequence helps the model capture short-term movement direction better than predicting only one future point.

3.5. Task 1 Cross-Validation

Time-series CV with 3 folds (expanding window) was run before the final experiment. CV results show fold-to-fold variation in MAE and R², reflecting the fact that different time periods in the Nasdaq data have different

volatility regimes. A model trained on a bull-market period may underperform when evaluated on a correction period. This instability is expected and reported transparently.

Task 1 Time-Series Cross-Validation Results:											
	Fold	Train Rows	Validation Rows	MAE	MSE	RMSE	MAPE	sMAPE	R2	Directional Accuracy	
0	1	1116	1116	4.189321	34.568459	5.879495	29.865240	37.707606	-0.189866		42.690583
1	2	2232	1116	3.196073	18.545224	4.306417	15.665344	13.711034	0.878242		45.919283
2	3	3348	1116	2.455315	11.287983	3.359759	18.623312	17.650739	0.702829		45.560538

Table 3.3: Task 1 Time-Series Cross-Validation Results

The model performs **poorly in Fold 1 but improves strongly in Folds 2 and 3**. This suggests that **the earliest training window did not contain enough representative market behavior for the validation period**. As more historical data becomes available in later folds, the model generalizes better, reaching positive R^2 and lower error.

Directional accuracy remains around **42–46%**, meaning **the model is better at forecasting price levels than predicting daily up/down direction**. This is **acceptable for Task 1 because the task is price regression**, while **trading direction is handled separately in Task 3**.

4. Task 2 - Vietnam Stock Price Prediction

4.1. Dataset Challenges

The Vietnam dataset uses a different file naming convention ([TICKER-EXCHANGE-History.csv](#), e.g., [ACB-VNINDEX-History.csv](#)) and different date formats. A custom resolver handles both the exchange suffix in filenames and multi-format date parsing. This reflects a real-world data engineering challenge — publicly available financial datasets are rarely clean, and robust parsing is necessary before any modeling.

The Vietnam feature set uses Close (not Adjusted Close) as the target, since many Vietnam files do not include an Adjusted Close column. The same 10 features are computed.

4.2. Ticker Selection Strategy

For Vietnam, the selection strategy switches from [longest_history](#) to [stable_active](#), which ranks tickers by a composite stability score that penalizes return volatility, spike frequency, and flat-price days. This is more appropriate for Vietnamese small-cap stocks, which can have long histories but with extended periods of trading suspensions (flat prices) or sudden price corrections (spikes) that would corrupt a time-series model's training signal.

4.3. Vietnam Model Parameters

The Vietnam model uses a **slightly lower dropout rate (0.25 vs 0.30 for Nasdaq)** and a **small recurrent dropout of 0.01**. This was applied **because Vietnamese stock data tends to have shorter usable histories** after filtering, meaning less training data, so slightly less regularization is appropriate to avoid under-fitting.

4.4. Additional Data Assessment

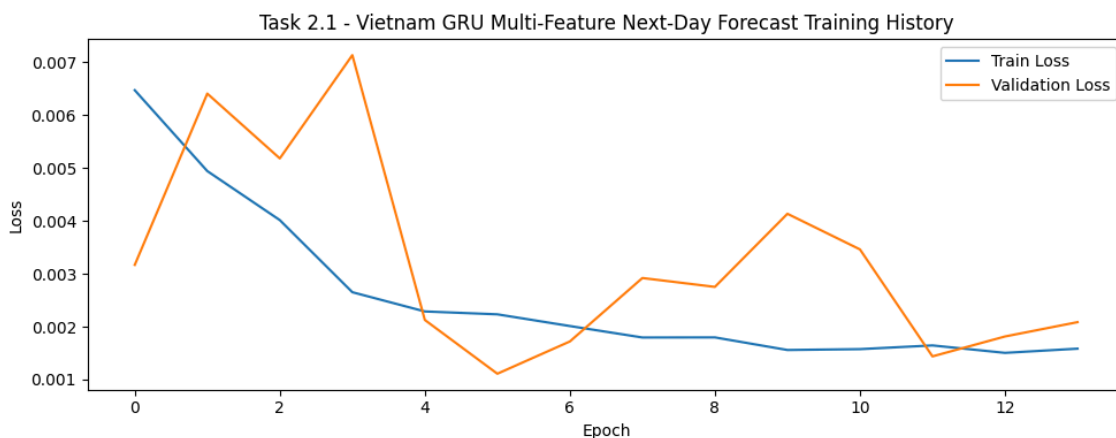
The Vietnam dataset also includes dividend history, financial ratios, and industry analysis files. These were assessed and deliberately excluded from Tasks 2.1–2.3 for the following reasons:

- Dividend history is reported at irregular, event-driven intervals (not daily), making it incompatible with the daily sequence format required by GRU models without a complex alignment step.
- Financial ratios (P/E, P/B, ROE, etc.) are reported quarterly or annually. Carrying a quarterly value forward through 63 daily trading days introduces a stale-signal problem — the model would learn to associate old financial health data with future prices that may reflect entirely different company conditions.
- Industry analysis provides static sector classifications (e.g., Banking, Real Estate, Consumer). These are categorical labels that require embedding or one-hot encoding and provide no time-varying signal for sequence prediction.

Conclusion: For daily price prediction with a 30-day lookback, technical price-derived features are more informative than low-frequency fundamental data. The additional datasets would be more useful in a longer-horizon or cross-sectional model (such as Task 4).

4.5. Results

4.5.1 Task 2.1 - Vietnam GRU Multi-feature Next-day Forecasting



The training loss decreases steadily, **showing that the GRU learns from the training data**. The **validation loss is noisier than Task 1**, which **suggests the Vietnam stock series has more unstable short-term behavior or fewer smooth patterns**. However, validation loss eventually returns to a low range, so the model still converges.

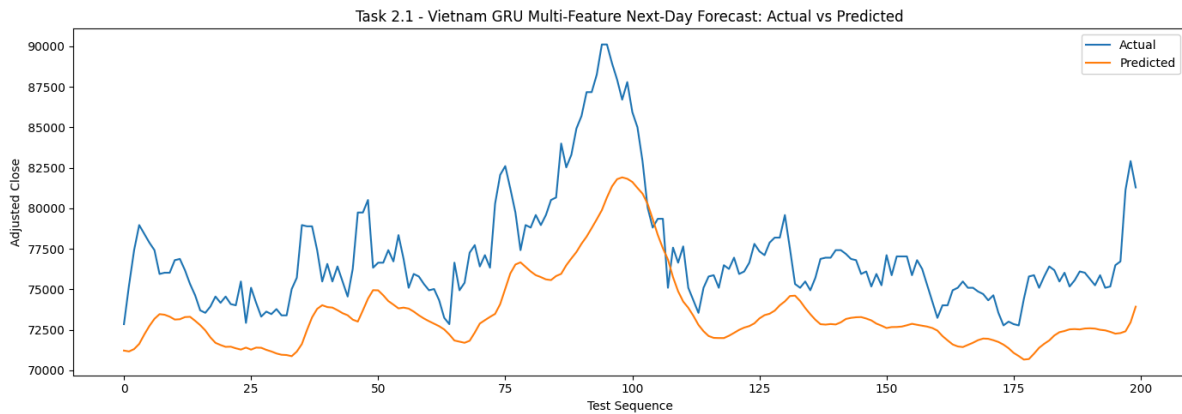


Figure 4.2: Task 2.1 Actual vs Predicted Vietnam Close Price

The **actual-vs-predicted** plot shows that the **GRU captures the general price level and broad trend**, but the **prediction curve is much smoother than the real price movement**. The model underestimates sharp upward spikes, especially around the middle and end of the test period. This means the GRU is useful for approximate price-level forecasting but weak at capturing sudden market jumps.

```
=====
Task 2.1 Vietnam - GRU vs Naive Baselines
=====
```

	Model	MAE	RMSE	sMAPE (%)	R2	Directional Accuracy (%)
0	GRU (your model)	4328.925467	4979.199875	5.528648	0.229111	44.204322
1	Persistent (naive)	1032.078431	1409.475163	1.306311	0.938229	42.829077
2	Mean baseline	4349.075917	5671.052869	5.461132	0.000000	7.858546

Table 4.1: Task 2.1 Vietnam GRU vs Naive Baselines

The baseline comparison **shows an important limitation: the persistent naive model outperforms the GRU on MAE, RMSE, sMAPE, and R²**. This happens **because stock prices are highly autocorrelated, so using today's price as tomorrow's prediction is a very strong baseline**. The GRU has slightly better directional accuracy than the persistent model, but its price-level error is worse.

4.5.2 Task 2.2 - Vietnam GRU Multi-feature 3rd Day Forecasting

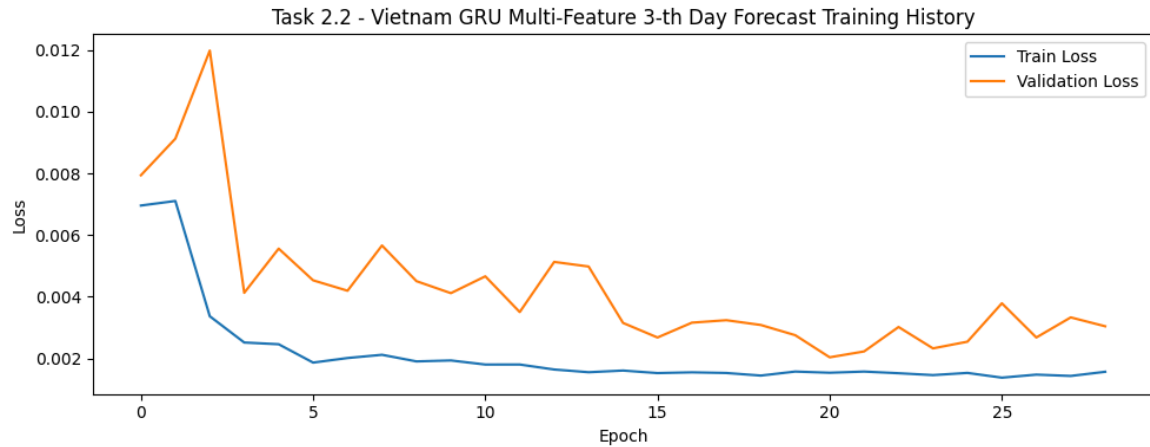


Figure 4.3: Task 2.2 GRU Training and Validation Loss

The training loss decreases and stabilizes, showing that the model learns the training data. However, the validation loss is unstable during early epochs and remains above the training loss, suggesting that the 3rd-day Vietnam forecast is harder to generalize than the next-day task.

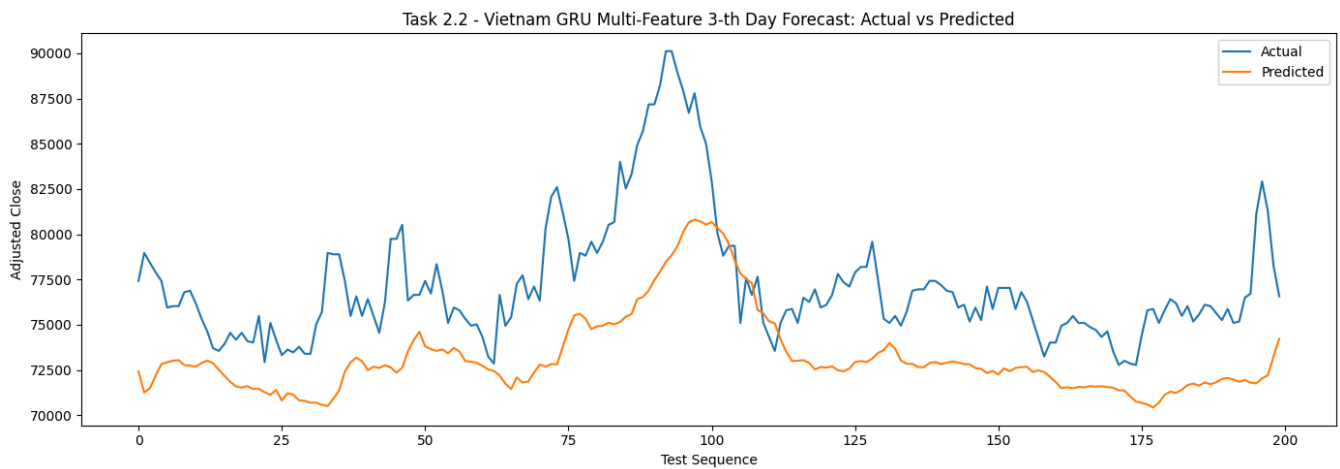


Figure 4.4: Task 2.2 Actual vs Predicted Vietnam Close Price

The actual-vs-predicted plot shows that the GRU follows the broad price movement but produces a much smoother curve than the actual prices. It underestimates sharp upward movements, especially around the large mid-test spike and the final jump. This indicates that the model captures slow trend behavior but struggles with sudden market changes.

```

Task 2.2 - Vietnam GRU Multi-Feature 3-th Day Forecast Metrics
MAE: 4688.4456
MSE: 29966561.6378
RMSE: 5474.1722
MAPE: 5.79%
sMAPE: 6.01%
R2: 0.0690
Directional Accuracy: 45.36%

```

Table 4.2: Task 2.2 Vietnam 3rd-Day Forecast Metrics

The metrics confirm this limitation. The model achieves **MAPE = 5.79%** and **sMAPE = 6.01%**, which are acceptable in percentage terms, but **$R^2 = 0.0690$** is weak. This means the model explains only a small portion of the test-set variance. Directional accuracy is **45.36%**, so it is not reliable for predicting short-term movement direction.

4.5.3 Task 2.3 - Vietnam GRU Multi-feature 3-Consecutive-Day Forecast

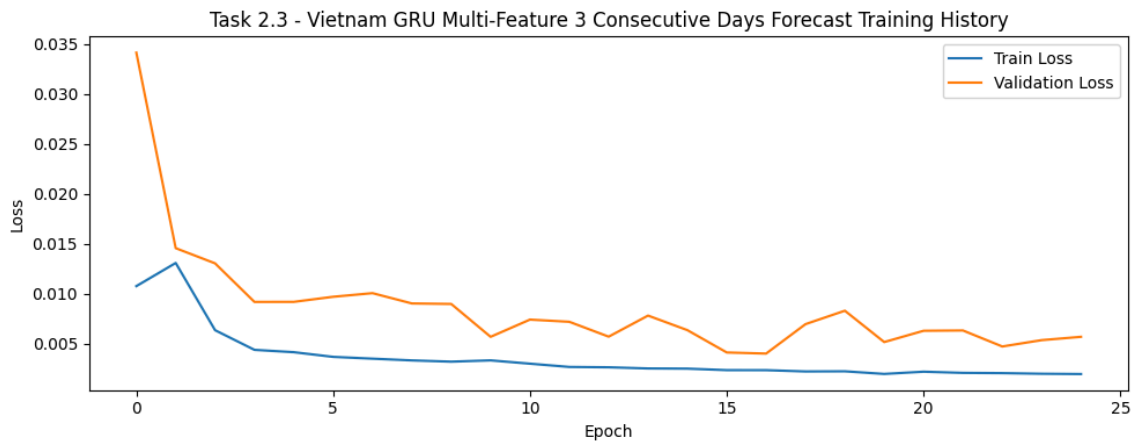


Figure 4.5: Task 2.3 GRU Training and Validation Loss

The training loss decreases and stabilizes, but the validation loss remains higher and more unstable than the training loss. This suggests that the model fits the training data but has difficulty generalizing to future Vietnam market movements, especially for multi-step prediction.

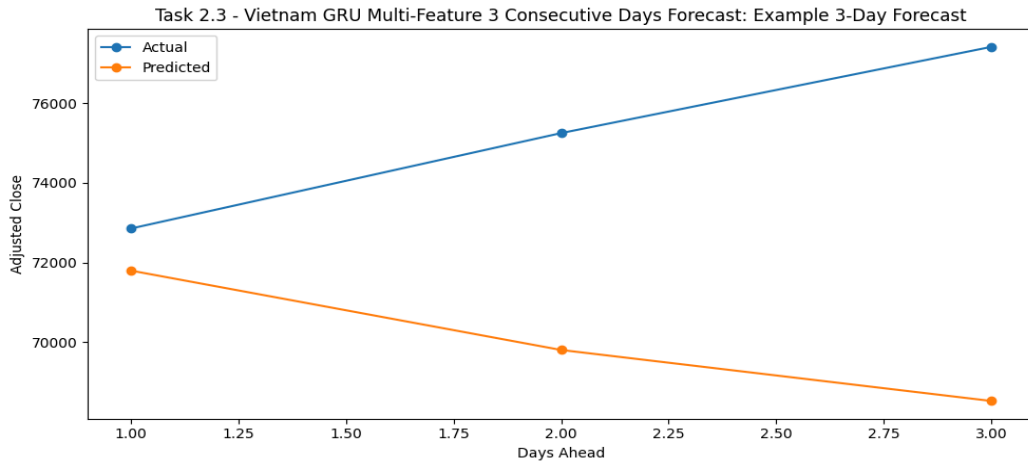


Figure 4.6: Task 2.3 Example 3-Day Vietnam Forecast

The **example 3-day forecast shows a clear mismatch between actual and predicted movement**. The actual price increases over the next three days, while the predicted sequence decreases. This indicates that the model struggles to capture short-term multi-day direction for this Vietnam stock.

```
Task 2.3 - Vietnam GRU Multi-Feature 3 Consecutive Days Forecast Metrics
MAE: 7199.6722
MSE: 63365111.9960
RMSE: 7960.2206
MAPE: 8.95%
sMAPE: 9.45%
R2: -0.9911
Directional Accuracy: 45.31%
```

Table 4.3: Task 2.3 Vietnam 3-Consecutive-Day Forecast Metrics

Task 2.3 has MAE = 7199.67, RMSE = 7960.22, MAPE = 8.96%, and $R^2 = -0.9911$. The negative R^2 means the model performs worse than a simple mean baseline for explaining test-set variance. Directional accuracy is 45.31%, so the model is also weak at predicting the direction of the 3-day movement.

Final Task 2 Summary:										
Task	Ticker	Model	Description	MAE	MSE	RMSE	MAPE	sMAPE	R2	Directional Accuracy
0	Task 2.1	VCB GRU	Vietnam multi-feature next-day forecast	4328.925467	2.479243e+07	4979.199875	5.345246	5.528648	0.229111	44.204322
1	Task 2.2	VCB GRU	Vietnam multi-feature 3-th day forecast	4688.445635	2.996656e+07	5474.172233	5.790545	6.010884	0.069040	45.364892
2	Task 2.3	VCB GRU	Vietnam multi-feature 3 consecutive days forecast	7199.672231	6.336511e+07	7960.220600	8.951610	9.448875	-0.991118	45.305318

Table 4.4: Overall Task 2 Vietnam Forecasting Performance Summary

The Task 2 summary shows that performance decreases as the forecast horizon becomes longer. Task 2.1 performs best, with the lowest MAE, RMSE, MAPE, and the highest R^2 . Task 2.2 is weaker but

still maintains a small positive R^2 . Task 2.3 performs worst, with a negative R^2 , showing that predicting a full 3-day sequence is much more difficult than predicting one future value. Directional accuracy stays around 44–45% across all tasks, meaning the model is better suited for approximate price-level forecasting than trading direction.

5. Task 3 - Trading Signal Identification

5.1. Problem Framing: Regression vs. Classification

Tasks 1 and 2 treat price prediction as a regression problem — the output is a continuous price value. Task 3 reframes the problem as binary classification: at each point in time, is the next 5 days a good time to buy (or sell)?

This framing has a key advantage: it is directly actionable. A trader does not need to know the exact future price; they need to know *whether* now is a good entry point. The model outputs a probability score, which can be thresholded to produce a binary signal.

5.2. Label Construction

Labels are generated from **future returns over a 5-day horizon** (`TASK3_SIGNAL_HORIZON = 5`). For each day t :

- **Buy Signal = 1** if $(\text{Close}_{\{t+5\}} / \text{Close}_t) - 1 \geq \text{buy_threshold}$
- **Sell Signal = 1** if $(\text{Close}_{\{t+5\}} / \text{Close}_t) - 1 \leq -\text{sell_threshold}$

The thresholds are **dynamically computed from the training set only** using quantiles:

- `buy_threshold` = 65th percentile of positive 5-day forward returns in training data
- `sell_threshold` = 35th percentile absolute value of negative 5-day forward returns

This design has two important properties. First, thresholds are data-driven, not hand-coded — they adapt to the selected ticker's actual return distribution. Second, computing thresholds only from training data prevents look-ahead bias: the model does not know anything about the magnitude of future returns in the test period.

New concept — class imbalance: Strong buy or sell conditions do not occur on most days. In practice, approximately 25–35% of days qualify as buy signals and a similar proportion as sell signals, meaning the classes are moderately imbalanced. This is handled by computing inverse-frequency class weights (`compute_class_weight`) and passing them to the model during training, so the loss function penalizes misclassification of the minority class more heavily.

5.3. Feature Set

The Task 3 feature set is substantially larger than Tasks 1–2, including:

- **Basic prices and returns:** Open, High, Low, Close, Volume, 1/3/5-day returns
- **Trend indicators:** SMA(5/10/20), EMA(12/26), MA(10/20)
- **MACD and signal line:** Captures momentum shifts — when the MACD crosses its signal line, it suggests a change in trend direction. The histogram (MACD minus signal) quantifies the strength of the signal.
- **RSI(14):** Relative Strength Index. Values above 70 indicate overbought conditions (sell signal candidate); values below 30 indicate oversold conditions (buy signal candidate). Including raw RSI as a feature lets the model learn these thresholds implicitly.
- **Bollinger Band position and width:** Measures where the current price sits relative to recent price volatility. A price near the lower band in a narrowing band environment often precedes a breakout.
- **ATR(14) ratio:** Average True Range normalized by price — measures intraday volatility independently of price level, useful as a risk proxy.
- **OBV change:** On-Balance Volume change — confirms whether price moves are supported by volume. A price rise on declining OBV suggests a weak move likely to reverse.
- **Volatility5/10, Momentum5/10, Price Range, Close Position**

Justification for manual feature engineering: The GRU model learns temporal patterns from the raw sequence, but including pre-computed technical indicators gives it an explicit "vocabulary" of market concepts that practitioners have refined over decades. Rather than requiring the model to learn RSI from scratch from raw prices, we provide RSI as an input — the model then learns *when* RSI levels matter in the context of other features.

5.4. Threshold Optimization on Validation Set

After training, the classification threshold (default 0.5) is optimized on the validation set by scanning values from 0.3 to 0.7 and selecting the threshold that maximizes F1 score. This is important because F1 balances precision and recall — with an imbalanced dataset, setting the threshold too low yields many false buy signals, while too high misses most real opportunities.

5.5. Evaluation: ROC and Precision-Recall Curves

Two evaluation curves are produced for each model:

- **ROC curve:** Plots True Positive Rate vs. False Positive Rate across all thresholds. AUC-ROC measures overall discriminative ability. An AUC above 0.60 is meaningful for financial signal detection; near 0.50 is random.
- **Precision-Recall curve:** More informative for imbalanced datasets. Precision = of all days the model flags as buy signals, what fraction actually yielded positive returns. Recall = of all genuinely good days to buy, what fraction the model caught. The no-skill baseline

is the positive class rate (e.g., 0.30 if 30% of days are buy days). A PR AUC above this baseline confirms the model adds value.

New concept — the precision-recall trade-off in trading: A high-precision, low-recall model catches fewer signals but makes fewer false trades — suitable for conservative investors who want high-confidence entry points. A high-recall, low-precision model catches more opportunities but generates more false alarms — suitable for high-frequency traders willing to accept more noise. The F1 score balances these, but the optimal operating point depends on transaction costs and risk tolerance.

5.6 Results

5.6.1 Task 3 - Signal Label Distribution

	Split	Rows	Buy Positive Ratio	Sell Positive Ratio
0	Train	2359	0.350042	0.350467
1	Validation	505	0.390000	0.228000
2	Test	506	0.331337	0.323353

Table 5.1: Task 3 Buy/Sell Signal Class Distribution by Split

The **buy and sell signal labels** are moderately imbalanced. **Buy positive ratios** are around **33–39%**, while **sell positive ratios** range from **22.8% to 35.0%**. This confirms that **accuracy alone is not enough for evaluation**, so **precision, recall, F1, ROC AUC, and PR AUC** are reported.

5.6.2 Task 3 Time-Series Cross-Validation

Task 3.1 Buy Signal – Time-Series CV											
Fold 1 done – F1=0.214, AUC=0.555											
Fold 2 done – F1=0.219, AUC=0.497											
Fold 3 done – F1=0.464, AUC=0.503											
	Fold	Signal	Train Rows	Val Rows	Positive Rate	F1	Precision	Recall	ROC AUC	Buy Threshold	Sell Threshold
0	1	Buy Signal	810	837	0.399044	0.213777	0.517241	0.134731	0.554731	0.010850	0.020207
1	2	Buy Signal	1652	837	0.393070	0.219114	0.470000	0.142857	0.497062	0.013854	0.015111
2	3	Buy Signal	2494	837	0.348865	0.464246	0.341977	0.722603	0.502532	0.015184	0.012232

Table 5.2: Task 3.1 Buy Signal Time-Series Cross-Validation Results

The buy signal CV results are unstable across folds. Fold 3 has the best F1 score at 0.4642, but Folds 1 and 2 are much weaker. ROC AUC stays close to 0.50, meaning the classifier has limited ranking ability across time periods. This suggests that buy signals are noisy and regime-dependent.

Task 3.2 Sell Signal – Time-Series CV
Fold 1 done – F1=0.398, AUC=0.541
Fold 2 done – F1=0.300, AUC=0.451
Fold 3 done – F1=0.177, AUC=0.547

Fold	Signal	Train Rows	Val Rows	Positive Rate	F1	Precision	Recall	ROC AUC	Buy Threshold	Sell Threshold
0	1 Sell Signal	810	837	0.244922	0.397980	0.250955	0.960976	0.540877	0.010850	0.020207
1	2 Sell Signal	1652	837	0.267622	0.300188	0.258900	0.357143	0.450507	0.013854	0.015111
2	3 Sell Signal	2494	837	0.310633	0.176991	0.379747	0.115385	0.546900	0.015184	0.012232

Table 5.2: Task 3.1 Sell Signal Time-Series Cross-Validation Results

The sell signal CV results are also inconsistent. F1 ranges from 0.1770 to 0.3980, while ROC AUC fluctuates around 0.45–0.55. This shows that the sell classifier has some weak signal in certain folds, but does not generalize consistently across time.

5.6.3 Task 3.1 Buy Signal Model

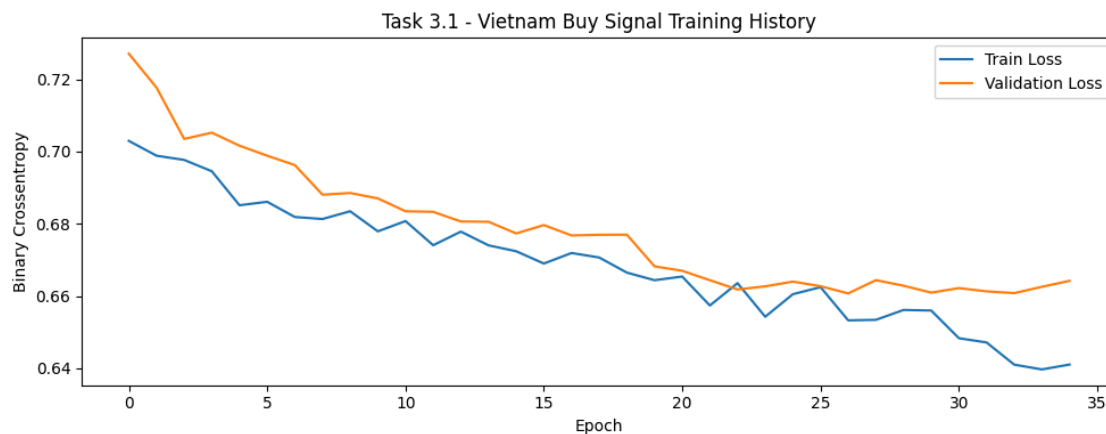


Figure 5.1: Task 3.1 Buy Signal Training and Validation Loss

The buy model loss decreases for both training and validation sets, suggesting stable convergence. Validation loss remains close to training loss, so there is no severe overfitting. However, good loss behavior does not translate into strong classification performance.

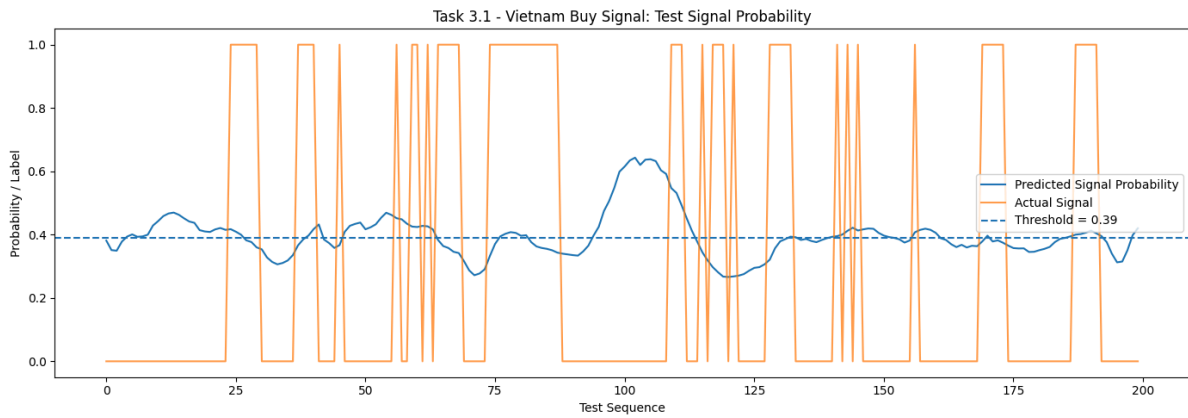


Figure 5.2: Task 3.1 Buy Signal Test Probability Curve

The predicted buy probabilities **mostly stay near the threshold region instead of clearly separating buy and non-buy periods**. This explains **the weak test performance and many false predictions**.

5.6.4 Task 3.2 Sell Signal Model

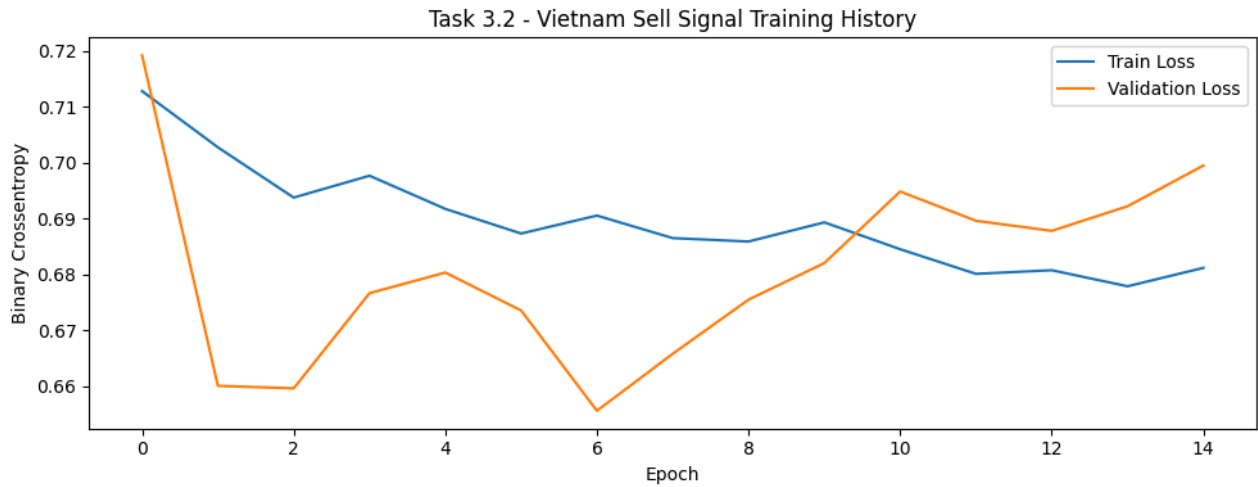
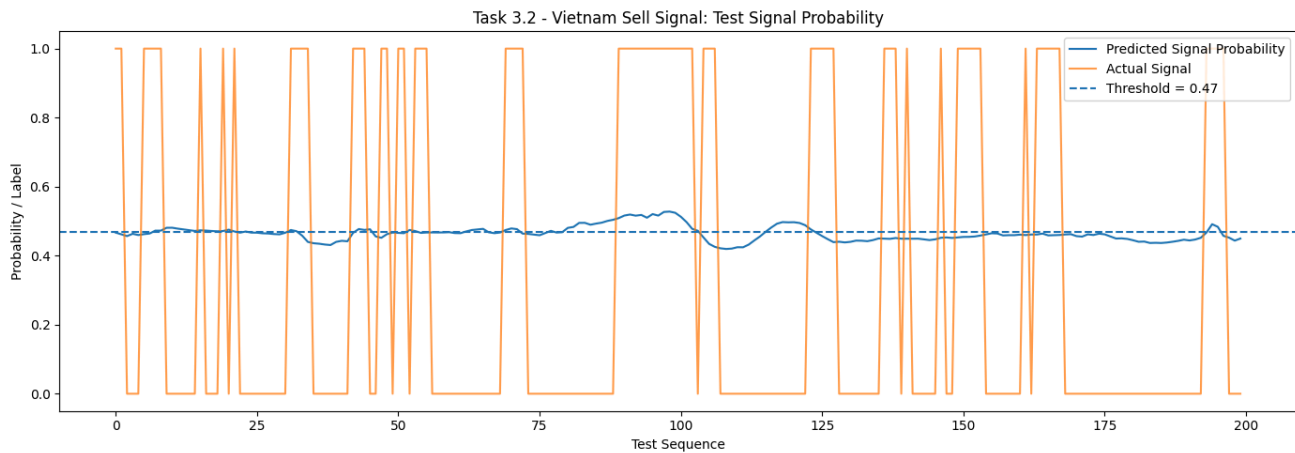


Figure 5.4: Task 3.2 Sell Signal Training and Validation Loss

The sell model training loss decreases, but validation loss starts increasing after early epochs. This suggests mild overfitting or unstable validation behavior. The model learns some training patterns, but generalization is limited.



The sell model performs better than the buy model. On the test set, it achieves accuracy = 51.70%, precision = 35.82%, recall = 62.35%, F1 = 0.4550, ROC AUC = 0.5511, and PR AUC = 0.3410. This indicates a weak but usable sell signal, especially because ROC AUC and PR AUC are slightly above the no-skill baseline.

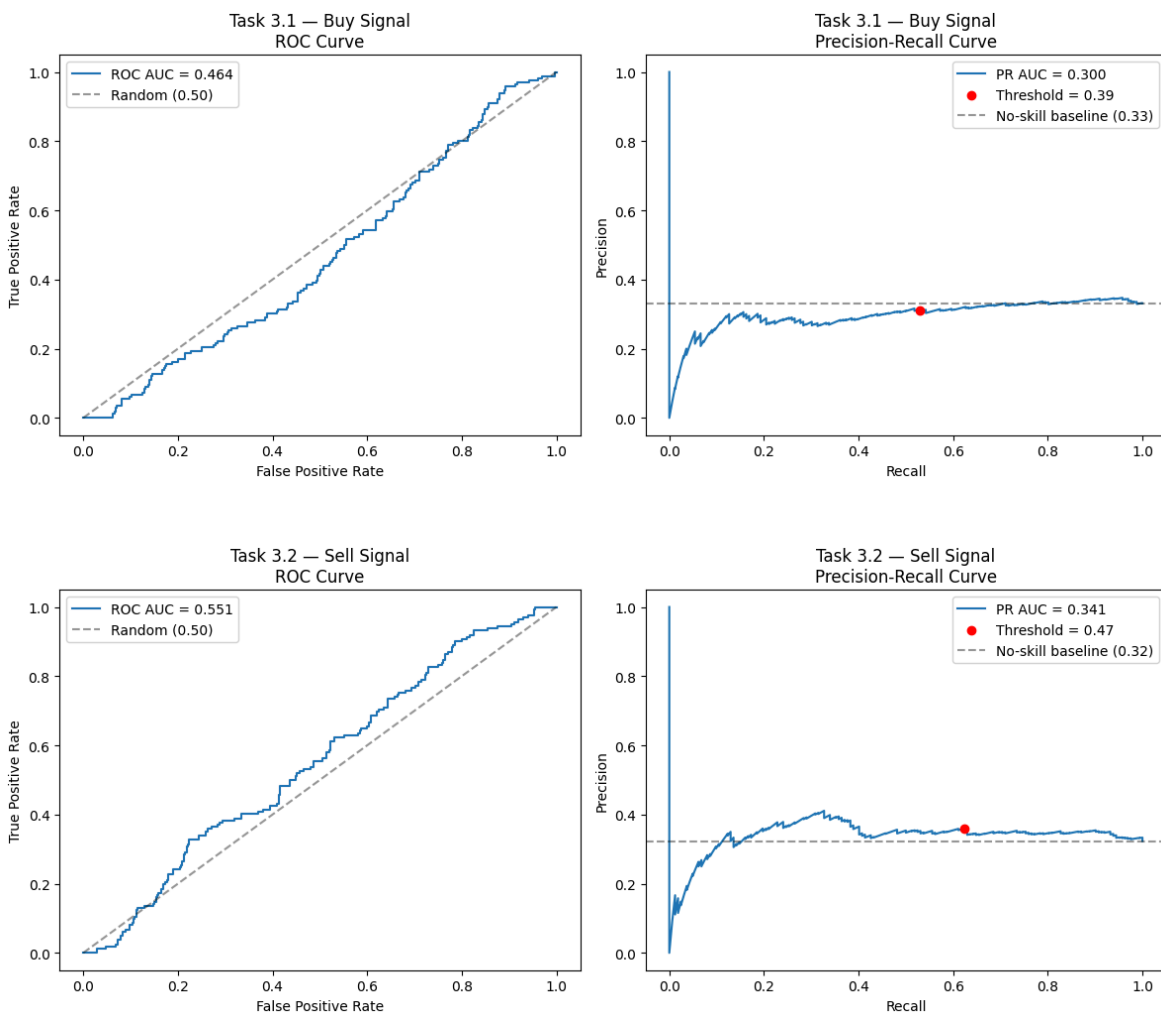


Figure 5.6: Task 3.2 Buy/Sell Signal ROC and Precision-Recall Curves

The ROC curve is close to or slightly below the random baseline, and the PR curve is near the no-skill baseline. This confirms that the buy classifier does not reliably distinguish profitable buy opportunities in the test period.

The sell ROC curve is slightly above the random baseline, and the PR curve is slightly above the no-skill baseline. This suggests the sell model has modest predictive value, although the signal is not strong enough to be used alone for trading decisions.

Final Task 3 Summary:												
Task	Signal	Model	Threshold	Accuracy	Precision	Recall	F1	ROC AUC	PR AUC	Positive Ratio	Predicted	Positive Ratio
0	Task 3.1	Buy	Task 3.1 - Vietnam Buy Signal Test	0.39	0.453094	0.309859	0.530120	0.391111	0.464215	0.299501	0.331337	0.566866
1	Task 3.2	Sell	Task 3.2 - Vietnam Sell Signal Test	0.47	0.516966	0.358156	0.623457	0.454955	0.551094	0.341007	0.323353	0.562874

Table 5.6: Final Task 3 Buy/Sell Signal Classification Summary

The final Task 3 summary shows that the sell signal model performs better than the buy signal model. The buy model has accuracy = 45.31%, F1 = 0.3911, ROC AUC = 0.4642, and PR AUC = 0.2995, indicating weak predictive ability. Its ROC AUC below 0.50 means it does not reliably separate buy and non-buy cases.

The sell model achieves accuracy = 51.70%, F1 = 0.4550, ROC AUC = 0.5511, and PR AUC = 0.3410. This is still modest, but it is stronger than the buy model and slightly above random ranking performance. The higher recall of 62.35% suggests the sell model is better at detecting potential sell-risk periods, although precision remains low.

Both models predict positives more often than the true positive ratio. The true buy and sell ratios are around 33% and 32%, but the predicted positive ratios are around 56%. This means the models are still somewhat aggressive in predicting signals, which increases recall but creates more false positives.

Overall, Task 3 satisfies the trading signal identification requirement, but the results should be interpreted as exploratory. **The buy classifier is weak, while the sell classifier shows modest predictive value. These models should not be used as standalone trading systems; they are better treated as signal features** that could support the portfolio analysis in Task 4.

6. Task 4 — Portfolio Composition, Risk Management, and Optimization

6.1 Universe Construction

For Task 4, the model is trained on a **universe of 40 stocks** simultaneously, rather than a single ticker. This multi-ticker approach is essential because portfolio construction requires comparing and ranking companies against each other. The 40 candidates are drawn from the eligible tickers in Vietnam, sorted by lowest spike rate and longest history.

The minimum row requirement for Task 4 is $\max(120, 60 + 20 + 30) = 120$ rows, ensuring each ticker has enough history for a 60-day lookback plus a 20-day forecast horizon plus a buffer.

6.2 Extended Feature Engineering

Task 4 uses 15 features per ticker, computed at the individual stock level:

- **Return1, Return5, Return20:** Short-, medium-, and longer-term momentum signals.
- **SMA10/20/50 Ratio:** Price relative to its moving averages (deviation from trend), rather than raw MA values. This makes the feature scale-independent across stocks of different price magnitudes.
- **Volatility10/20/60:** Multi-scale volatility from 10 to 60 days, capturing short-term noise vs. structural risk.
- **Momentum10, Momentum20:** Price 10 or 20 days ago relative to today — captures trend persistence.
- **RSI14:** Momentum oscillator for overbought/oversold detection.
- **Volume Change:** Percentage change in daily volume — confirms or questions price moves.
- **Dollar Volume:** Close $\times$ Volume, a liquidity proxy that is scale-aware.
- **Price Range:** (High – Low) / Close — intraday volatility as a fraction of price.
- **Future Return (target):** $(\text{Close}_{t+20} / \text{Close}_t) - 1$ — the 20-day forward return the model is trained to predict.

New concept — SMA Ratio vs. raw SMA: Using $\text{Close} / \text{SMA} - 1$ instead of raw SMA values removes price-level dependence. A stock at 5,000 VND and one at 50,000 VND have very different raw SMA values, but their SMA ratios (how far price deviates from trend) are comparable. This allows a single shared GRU to learn patterns across stocks of different absolute price levels.

6.3. Shared GRU Return Forecasting Model

A single GRU model is trained on all 40 tickers' sequences simultaneously. The architecture is identical to Tasks 1–2, but adds a `Dense(16, relu)` intermediate layer before the final `Dense(1)` output, providing more expressivity for the richer 15-feature input.

The model learns to predict 20-day forward returns. Using a **shared model across tickers** is a design choice with trade-offs:

- **Advantage:** Cross-ticker learning. Patterns observed in one stock (e.g., high RSI + declining volume $\rightarrow$ reversion) can generalize to others, giving the model more training data than a per-ticker model would have.
- **Limitation:** A single model cannot capture company-specific characteristics (e.g., seasonal patterns in retailers vs. banks). This is an acceptable simplification given the project scope.

The StandardScaler used for Task 4 is fit on the combined training data from all tickers. This means the scaler's mean and variance reflect the distribution across the entire universe, which is appropriate for a model that must generalize across stocks.

6.4. Profitability Scoring (Task 4.1)

After the GRU generates a predicted 20-day return for each ticker, a **composite profitability score** is computed:

$$\begin{aligned}\text{Profitability Score} = & 0.45 \times \text{Predicted Return Score} \\ & + 0.25 \times \text{Momentum Score (20D trailing return)} \\ & + 0.20 \times \text{Sharpe Score (60D risk-adjusted return)} \\ & + 0.10 \times \text{Liquidity Score (median dollar volume)}\end{aligned}$$

Each component is min-max normalized to [0, 1] across the universe. This scoring system combines forward-looking model output (45% weight) with backward-looking evidence (55% weight), reflecting the practical reality that GRU predictions alone are noisy — recent momentum and risk-adjusted performance provide corroborating evidence.

The **Sharpe ratio** component is computed over a 60-day rolling window: $\text{mean_daily_return} / \text{daily_return_std} \times \sqrt{252}$. Including Sharpe instead of raw return rewards stocks that generate consistent gains relative to their volatility — a more investor-relevant measure than raw return.

6.5. Risk Scoring (Task 4.2)

The risk score identifies companies to exclude:

$$\begin{aligned}\text{Risk Score} = & 0.35 \times \text{Volatility Risk (60D return std)} \\ & + 0.30 \times \text{Downside Risk (60D downside volatility)} \\ & + 0.25 \times \text{Drawdown Risk (120D max drawdown magnitude)} \\ & + 0.10 \times \text{Liquidity Risk (inverse of dollar volume)}\end{aligned}$$

New concept — downside volatility vs. total volatility: Standard deviation penalizes both upside and downside fluctuations equally. But investors care primarily about downside risk. Downside volatility is computed as the standard deviation of only the negative daily returns, providing a purer measure of loss risk. A stock that frequently gains 3% and rarely loses 0.5% has high total volatility but very low downside volatility.

New concept — maximum drawdown: The maximum drawdown measures the largest peak-to-trough decline in the price series over the 120-day window. A drawdown of -30% means the stock fell 30% from its recent high at some point — a severe loss for a buy-and-hold investor. This metric captures tail risk that volatility alone does not.

A stock is flagged as **Exclude** if its Risk Score exceeds 0.70 (the 70th percentile of the universe), or if its non-zero volume rate falls below 30% (an illiquid stock that cannot be reliably traded).

6.6. Portfolio Construction (Task 4.3)

Two portfolios of 8 stocks each are constructed from the investable universe (stocks not flagged as Exclude):

Risk-Taking Portfolio:

- Selection: Ranked by $\text{Profitability Score} - 0.25 \times \text{Risk Score}$ (light risk penalty, emphasizes expected return).
- Weighting: Proportional to predicted return (stocks with higher predicted return receive larger allocation).
- Rationale: An aggressive investor wants maximum expected upside while accepting higher volatility.

Prudent Portfolio:

- Selection: Ranked by $\text{Profitability Score} - 0.85 \times \text{Risk Score}$ (heavy risk penalty, strongly penalizes high-risk stocks).
- Weighting: **Inverse-volatility weighting** — each stock receives a weight proportional to $1/\sigma$, where σ is its 60-day return standard deviation. This allocates more capital to lower-risk stocks.
- Rationale: Inverse-volatility weighting is a well-established portfolio construction technique (related to the risk parity framework) that equalizes each stock's risk contribution to the portfolio. It is not arbitrary — it can be derived from the assumption that all stocks have equal expected Sharpe ratios.

Mean-Variance Optimization: A third portfolio is constructed using Markowitz mean-variance optimization via `scipy.optimize.minimize`. The objective function maximizes $\mu^T w - (\lambda/2) w^T \Sigma w$, where μ is the vector of GRU predicted returns, Σ is the historical return covariance matrix computed from aligned daily return series, and $\lambda=2.0$ is the risk aversion parameter. The optimization is subject to long-only constraints ($w \geq 0$) and full investment (sum of weights = 1).

New concept — risk aversion parameter λ : In the Markowitz framework, λ controls the trade-off between expected return and variance. $\lambda=0$ gives the maximum-return portfolio (concentrated in the highest-return stock). As λ increases, the optimizer increasingly penalizes variance, moving toward the minimum-variance portfolio. $\lambda=2.0$ is a moderate choice representing a balanced investor.

6.7. Portfolio Backtest

The portfolios are evaluated over the test period using a buy-and-hold simulation: each day, the portfolio's return equals the weighted sum of each stock's daily return. The following performance metrics are computed:

- **Total Return:** Cumulative percentage gain or loss over the entire test period.
- **Annualized Return:** Scaled to a one-year equivalent using $(1 + \text{total_return})^{(252/n_days)} - 1$.

- **Sharpe Ratio:** $\text{mean_daily_return} / \text{std_daily_return} \times \sqrt{252}$. Values above 1.0 indicate risk-adjusted performance comparable to well-managed equity funds; above 2.0 is exceptional.
- **Maximum Drawdown:** The worst peak-to-trough decline during the test period — a measure of downside scenario risk.
- **Calmar Ratio:** $\text{annualized_return} / |\text{max_drawdown}|$. Measures how much return is generated per unit of drawdown risk. Higher is better.
- **Positive Days:** Percentage of trading days where the portfolio gained value, indicating consistency.

An **equal-weight benchmark** (all investable stocks weighted equally) is computed alongside the two portfolios to test whether the selection and weighting methodology adds value over a naive diversification strategy.

6.8 Results

6.8.1 Shared GRU Return Model

Task 4 shared GRU return model test metrics:					
	MAE	MSE	RMSE	R2	Directional Accuracy
0	0.094041	0.018566	0.136256	-0.025069	49.633669

Table 6.1: Task 4 Shared GRU Return Model Test Metrics

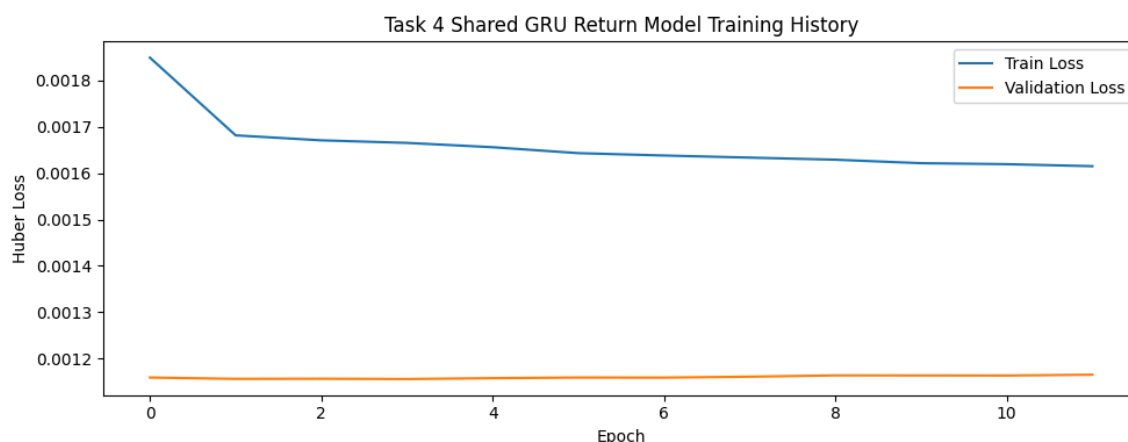


Figure 6.1: Task 4 Shared GRU Return Model Training History

The shared GRU return model has weak standalone predictive performance, with $R^2 = -0.0251$ and directional accuracy = 49.63%. This means the model is not strong enough to be used alone for portfolio decisions. Therefore, Task 4 uses the GRU predicted return as only one component in a hybrid stock scoring framework.

6.8.2 Predicted Return Candidates

	Ticker	Prediction Date	Current Close	Predicted Return	Predicted Price
18	CLC	2023-02-28	34900.0	0.012318	35329.909583
27	DHG	2023-02-28	96300.0	0.009195	97185.521453
7	DHA	2023-02-28	36700.0	0.008667	37018.092388
25	TDH	2023-02-28	3380.0	0.008667	3409.295702
23	PAC	2023-02-28	31400.0	0.008667	31672.155340
39	DRC	2023-02-28	22250.0	0.008667	22442.848927
28	HMC	2023-02-28	10500.0	0.008667	10591.007359
35	VTB	2023-02-28	11900.0	0.008667	12003.141673
34	LGC	2023-02-28	56900.0	0.008667	57393.173211
36	BMC	2023-02-28	13300.0	0.008667	13415.275988
14	BMP	2023-02-28	59800.0	0.007035	60220.708136
38	PJT	2023-02-27	8800.0	0.006136	8853.993900
21	RAL	2023-02-28	89000.0	0.005705	89507.712251
6	HAS	2023-02-27	6610.0	0.005558	6646.737893
5	SAV	2023-02-28	14450.0	0.005019	14522.526229
22	FMC	2023-02-28	36000.0	0.004992	36179.713503
30	DMC	2023-02-28	45000.0	0.004977	45223.957831
2	TMS	2023-02-28	53300.0	0.004841	53558.046785
29	VIP	2023-02-28	10150.0	0.004613	10196.823056
8	MHC	2023-02-28	3730.0	0.003934	3744.672266

Table 6.2: Top Predicted Return Stocks from Shared GRU Model

The predicted-return table identifies stocks with the highest model-estimated upside over the forecast horizon. **CLC has the highest predicted return, followed by DHG and DHA.** However, because the GRU return model is weak on its own, these predictions are combined with historical profitability and risk metrics before portfolio selection.

6.8.3 Profitability and Risk Scoring

Task 4 stock profitability and risk scores:																					
Ticker	Trailing Return 280	Trailing Return 680	Volatility 280	Volatility 680	Downside Volatility 680	Max Drawdown 1280	Sharpe 680	Nonzero Volume Rate	Median Dollar Volume	...	Sharpe Score	Liquidity Score	Volatility Risk	Downside Risk	Drawdown Risk	Liquidity Risk	Profitability Score	Risk Score	Risk Adjusted Score	Risk Flag	
0	DHG	0.020127	0.160241	0.010710	0.012579	0.004831	-0.131522	3.226981	1.000000	7.265570e+08	...	0.889573	0.001897	0.088595	0.010385	0.047049	0.998103	0.732737	0.145697	0.645320	Keep
1	CLC	0.090386	0.103592	0.014953	0.016025	0.012923	-0.162168	1.754263	0.958333	1.088588e+08	...	0.640964	0.000281	0.166493	0.263855	0.102316	0.999719	0.745506	0.262860	0.591778	Keep
2	DMC	-0.021739	0.071429	0.005644	0.011226	0.007359	-0.184368	1.714277	0.991667	2.187352e+08	...	0.640377	0.000569	0.058012	0.089570	0.142350	0.999431	0.623168	0.182706	0.513545	Keep
3	LGC	0.236957	0.306844	0.039490	0.036634	0.026745	-0.258872	2.219581	0.833333	8.445200e+06	...	0.723619	0.000019	0.632378	0.696848	0.276709	0.999981	0.812888	0.599562	0.453151	Keep
4	SAV	-0.043046	0.156000	0.016320	0.017030	0.010710	-0.340369	2.386929	0.983333	1.104400e+08	...	0.751187	0.000285	0.188230	0.194532	0.423679	0.999715	0.634062	0.330481	0.435773	Keep
5	SJD	-0.032258	0.003344	0.006473	0.008659	0.004500	-0.248619	0.169284	1.000000	7.335974e+08	...	0.385862	0.001916	0.000000	0.000000	0.258219	0.998084	0.534243	0.164363	0.435625	Keep
6	DHA	-0.021333	0.361074	0.015366	0.025195	0.018087	-0.400011	3.440745	1.000000	1.159835e+09	...	0.924788	0.003031	0.373789	0.425615	0.531235	0.999969	0.712700	0.491016	0.418090	Keep
7	RAL	-0.062474	0.105690	0.015089	0.018982	0.007979	-0.308333	1.544767	1.000000	1.635016e+09	...	0.612453	0.004274	0.233345	0.106990	0.365926	0.995720	0.591210	0.305417	0.407960	Keep
8	BMP	-0.080000	0.087273	0.021030	0.022151	0.012772	-0.207055	1.171802	1.000000	3.944725e+09	...	0.551012	0.010317	0.304991	0.259123	0.183264	0.989683	0.592478	0.320268	0.394917	Keep
9	DRC	-0.040948	0.116576	0.025398	0.019630	0.012332	-0.463116	1.840613	1.000000	6.889567e+09	...	0.628242	0.018022	0.247990	0.245343	0.845036	0.981978	0.644203	0.418956	0.392289	Keep
10	HMC	-0.009434	0.151316	0.012067	0.020463	0.020144	-0.426712	1.985496	1.000000	5.228911e+08	...	0.685057	0.001364	0.268831	0.490071	0.579387	0.998636	0.671070	0.485122	0.379997	Keep
11	TMS	-0.020221	-0.083815	0.013117	0.015672	0.010851	-0.194169	-1.354089	0.991667	3.737263e+08	...	0.134909	0.000974	0.158517	0.198972	0.160025	0.999026	0.521761	0.255081	0.368712	Keep
12	LBM	0.002336	0.021429	0.018720	0.024618	0.012744	-0.135667	0.417166	0.983333	8.128675e+07	...	0.426697	0.000209	0.360747	0.054525	0.999791	0.556457	0.317346	0.366049	Keep	
13	TYA	-0.054688	0.025424	0.010688	0.014755	0.011903	-0.280000	0.565300	1.000000	2.534975e+08	...	0.451100	0.000660	0.137789	0.231926	0.314810	0.999340	0.532205	0.296441	0.354341	Keep
14	FMC	-0.051383	0.132075	0.015233	0.022111	0.014710	-0.418605	1.657959	1.000000	1.957702e+09	...	0.631100	0.005118	0.304079	0.319842	0.564766	0.994882	0.605749	0.443060	0.339913	Keep
15	VIP	-0.009756	0.309677	0.033319	0.031635	0.021751	-0.309019	2.507590	1.000000	1.707150e+09	...	0.771064	0.004463	0.519389	0.540406	0.367142	0.995537	0.653052	0.535240	0.331908	Keep
16	FPT	-0.035928	0.083445	0.008690	0.011714	0.007814	-0.231935	1.902346	1.000000	7.935898e+10	...	0.671369	0.207628	0.069063	0.103825	0.228131	0.792372	0.440496	0.191589	0.325542	Keep
17	PAC	0.011272	0.049465	0.024578	0.031994	0.024481	-0.250343	0.650339	0.925000	5.135040e+07	...	0.465109	0.000131	0.527490	0.625925	0.261328	0.999899	0.638239	0.537718	0.315608	Keep
18	TDH	0.056250	0.076433	0.030558	0.028698	0.017773	-0.595082	0.902562	1.000000	3.060064e+08	...	0.508659	0.000797	0.452970	0.415788	0.883020	0.999203	0.671120	0.603951	0.308750	Keep
19	PJT	-0.022222	0.097257	0.027808	0.030827	0.021691	-0.312150	1.036829	0.925000	6.759250e+06	...	0.528777	0.000014	0.501095	0.538528	0.372788	0.999998	0.610635	0.530137	0.292553	Keep

Table 6.3: Task 4 Stock Profitability and Risk Score Ranking

The stock scoring table combines predicted return, trailing return, Sharpe ratio, liquidity, volatility, downside volatility, and drawdown. Stocks such as DHG, CLC, DMC, LGC, and SAV receive strong

risk-adjusted scores because they balance return potential with acceptable risk. This hybrid score is more reliable than using the GRU prediction alone.

6.8.4 Profitable Stock Selection

Task 4.1 Profitable stock selection:

	Ticker	Current Close	Predicted Return	Predicted Price	Trailing Return 280	Trailing Return 600	Sharpe 600	Profitability Score	Risk Score	Risk Adjusted Score
3	LGC	56900.0	0.008667	57393.173211	0.236957	0.306844	2.219581	0.812888	0.599562	0.453151
1	CLC	34900.0	0.012318	35329.909583	0.090386	0.103592	1.754263	0.749566	0.262980	0.591778
0	DHG	96300.0	0.009195	97185.521453	0.020127	0.160241	3.226981	0.732737	0.145697	0.645320
6	DHA	36700.0	0.008667	37018.092388	-0.021333	0.361074	3.440745	0.712700	0.491016	0.418090
18	TDH	3380.0	0.008667	3409.295702	0.056250	0.076433	0.902562	0.671120	0.603951	0.308750
10	HMC	10500.0	0.008667	10591.007359	-0.009434	0.151316	1.985496	0.671070	0.485122	0.379997
15	VIP	10150.0	0.004613	10196.823056	-0.009756	0.309677	2.507590	0.653052	0.535240	0.331908
9	DRC	22250.0	0.008667	22442.848927	-0.040948	0.116576	1.640613	0.644203	0.419856	0.392289
17	PAC	31400.0	0.008667	31672.155340	0.011272	0.049465	0.650339	0.638239	0.537718	0.315608
4	SAV	14450.0	0.005019	14522.526229	-0.043046	0.156000	2.386929	0.634062	0.330481	0.435773
2	DMC	45000.0	0.004977	45223.957831	-0.021739	0.071429	1.714277	0.623168	0.182706	0.513545
20	BMC	13300.0	0.008667	13415.275988	-0.076389	0.117647	1.280345	0.611388	0.564939	0.272425
19	PJT	8800.0	0.006136	8853.993900	-0.022222	0.097257	1.036829	0.610635	0.530137	0.292553
14	FMC	36000.0	0.004992	36179.713503	-0.051383	0.132075	1.657959	0.605749	0.443060	0.339913
8	BMP	59800.0	0.007035	60220.708136	-0.080000	0.087273	1.171802	0.592478	0.329268	0.394917

Table 6.4: Task 4.1 Selected Profitable Stocks

The profitable stock selection ranks companies by profitability score and risk-adjusted score. LGC has the highest profitability score, while DHG and CLC have stronger risk-adjusted profiles. These selected stocks form the candidate pool for portfolio construction.

6.8.5 Risky Stocks to Exclude

Task 4.2 Risky stocks to exclude:

	Ticker	Risk Score	Volatility 60D	Downside Volatility 60D	Max Drawdown 120D	Nonzero Volume Rate	Median Dollar Volume	Risk Flag
36	HAS	0.885424	0.049035	0.036423	-0.473594	0.725000	1.369000e+06	Exclude
39	SCD	0.828469	0.052897	0.032396	-0.363415	0.750000	9.325000e+06	Exclude
38	GIL	0.810495	0.042716	0.025933	-0.647272	1.000000	1.782816e+10	Exclude
34	MCP	0.802544	0.042036	0.026914	-0.610769	0.808333	7.517500e+06	Exclude
37	HBC	0.766220	0.040934	0.022440	-0.659950	1.000000	2.953458e+10	Exclude
33	ITA	0.754857	0.041345	0.023060	-0.602761	1.000000	9.097153e+09	Exclude
32	SMC	0.728408	0.036360	0.022239	-0.644724	1.000000	2.291284e+09	Exclude

Table 6.5: Task 4.2 Risky Stocks Excluded from Portfolio Construction

6.8.6 Portfolio Construction

Task 4.3 Risk-taking portfolio:									
Ticker	Risk Profile	Weight	Predicted Return	Profitability Score	Risk Score	Risk Adjusted Score	Volatility 60D	Max Drawdown 120D	
0	CLC	risk_taking	0.186136	0.012318	0.749566	0.262980	0.591778	0.016025	-0.162168
1	DHG	risk_taking	0.138948	0.009195	0.732737	0.145697	0.645320	0.012579	-0.131522
2	LGC	risk_taking	0.130968	0.008667	0.812888	0.599562	0.453151	0.036634	-0.258872
3	DHA	risk_taking	0.130968	0.008667	0.712700	0.491016	0.418090	0.025195	-0.400011
4	DRC	risk_taking	0.130968	0.008667	0.644203	0.419856	0.392289	0.019630	-0.463116
5	HMC	risk_taking	0.130968	0.008667	0.671070	0.485122	0.379997	0.020463	-0.426712
6	SAV	risk_taking	0.075841	0.005019	0.634062	0.330481	0.435773	0.017030	-0.340369
7	DMC	risk_taking	0.075202	0.004977	0.623168	0.182706	0.513545	0.011226	-0.184368
Task 4.3 Prudent portfolio:									
Ticker	Risk Profile	Weight	Predicted Return	Profitability Score	Risk Score	Risk Adjusted Score	Volatility 60D	Max Drawdown 120D	
0	SJD	prudent	0.203738	0.001258	0.534243	0.164363	0.435625	0.008659	-0.248619
1	DMC	prudent	0.157160	0.004977	0.623168	0.182706	0.513545	0.011226	-0.184368
2	DHG	prudent	0.140256	0.009195	0.732737	0.145697	0.645320	0.012579	-0.131522
3	TMS	prudent	0.112573	0.004841	0.521761	0.255081	0.368712	0.015672	-0.194169
4	CLC	prudent	0.110095	0.012318	0.749566	0.262980	0.591778	0.016025	-0.162168
5	SAV	prudent	0.103592	0.005019	0.634062	0.330481	0.435773	0.017030	-0.340369
6	RAL	prudent	0.092942	0.005705	0.591210	0.305417	0.407960	0.018982	-0.308333
7	BMP	prudent	0.079644	0.007035	0.592478	0.329268	0.394917	0.022151	-0.207055

Table 6.6: Task 4.3 Risk-Taking Portfolio Weights & Prudent Portfolio Weights

The risk-taking portfolio gives larger weights to stocks with stronger predicted return and profitability, such as CLC, DHG, LGC, DHA, DRC, and HMC. The prudent portfolio uses more conservative weighting and includes lower-risk names such as SJD, DMC, DHG, TMS, CLC, SAV, RAL, and BMP. This satisfies the requirement to distinguish between risk-taking and prudent investors.

6.8.7 Backtest Evaluation

Backtest starts: 2019-12-16 00:00:00 Task 4 Backtest Performance Summary:								
	Portfolio	Total Return (%)	Annualized Return (%)	Sharpe Ratio	Max Drawdown (%)	Calmar Ratio	Positive Days (%)	Trading Days
0	Risk-Taking	64.04	16.92	0.930	-36.45	0.464	57.0	798
1	Prudent (Inv-Vol)	56.77	15.26	0.936	-32.29	0.472	53.9	798
2	Equal-Weight Benchmark	60.14	16.03	0.964	-33.94	0.472	54.5	798

Table 6.8: Task 4 Portfolio Backtest Performance Summary

The backtest compares the risk-taking portfolio, prudent inverse-volatility portfolio, and equal-weight benchmark over the test period. The risk-taking portfolio achieves the highest total return at 64.04%, while the prudent portfolio has a lower drawdown at -32.29%. The equal-weight benchmark has the highest Sharpe ratio at 0.964, showing that diversification remains competitive. Overall, the risk-taking portfolio maximizes return, while the prudent portfolio better controls downside risk.

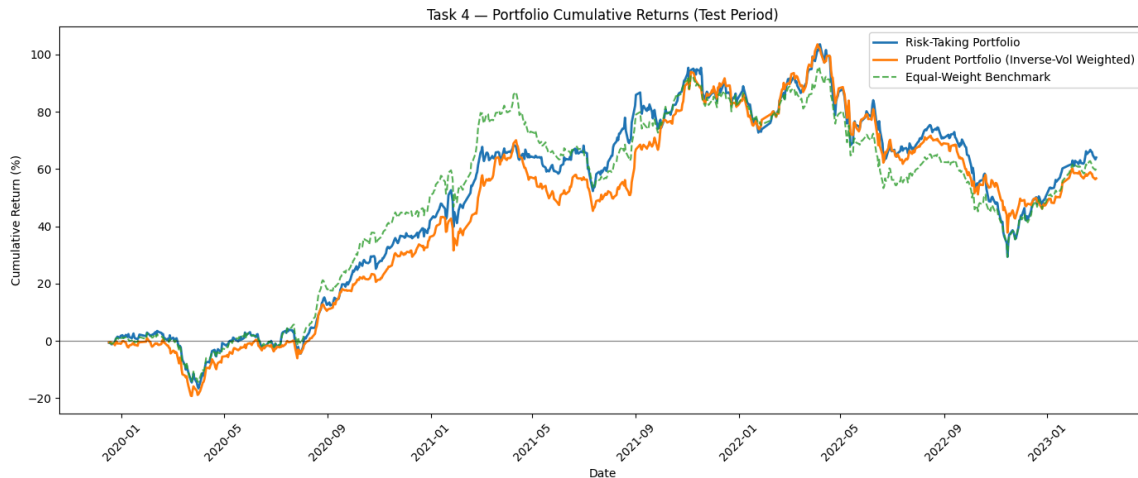


Figure 6.2: Task 4 Portfolio Cumulative Returns During Test Period

7. Task 5 - Industry standard for deployment and ease of use

7.1 Model Deployment

7.1.1 How to start the API Server?

Start the FastAPI:

```
cd "D:\Projects\Deep Learning Project\deployment"
python -m uvicorn api.api_app:app --reload
```

This command starts the FastAPI model server locally at `http://127.0.0.1:8000`. The API loads the exported model artifacts from `deployment/artifacts`, including the Keras models, scalers, metadata, and Task 4 portfolio files.

```
(.venv) PS D:\Projects\Deep Learning Project\deployment
> python -m uvicorn api.api_app:app --reload
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1778442507.677279 17580 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1778442509.081547 17580 port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to floating-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
INFO: Started server process [11632]
INFO: Waiting for application startup.
INFO: Application startup complete.
```

Figure 7.1: API runs without errors

7.1.2 How to send a prediction request?

A prediction request can be sent either through the FastAPI Swagger page or by sending an HTTP POST request to the prediction endpoint.

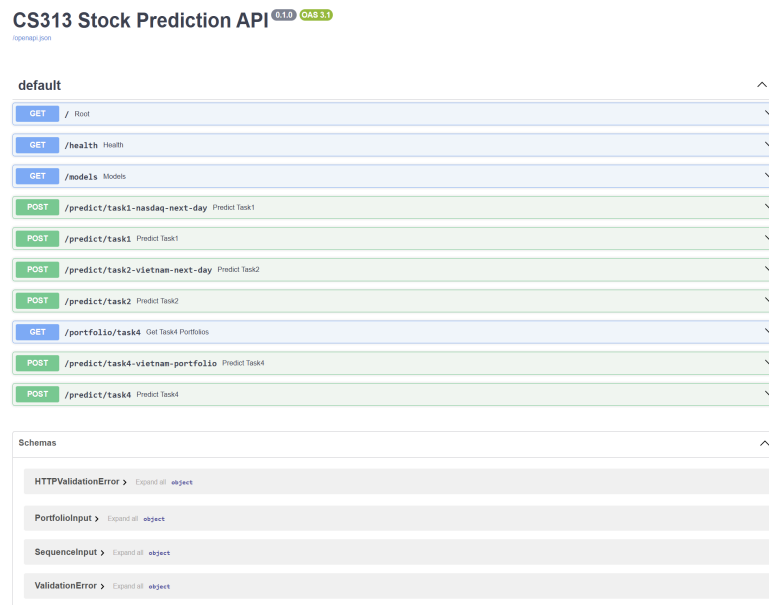


Figure 7.2: FastAPI Swagger Interface and Available Endpoints

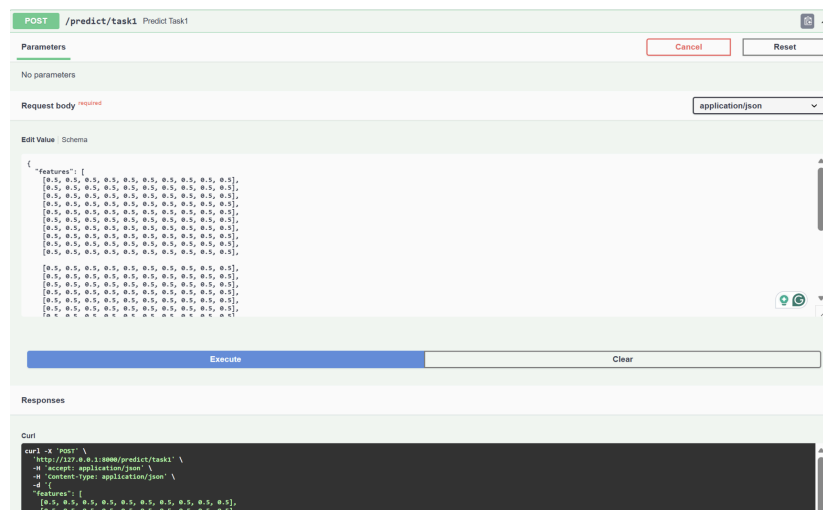
Our endpoints are:

- GET /health
- GET /models
- POST /predict/task1
- POST /predict/task2
- GET /portfolio/task4

The Swagger UI provides an interface for testing the endpoints: <http://127.0.0.1:8000/docs>

To test for the prediction request, proceed following steps:

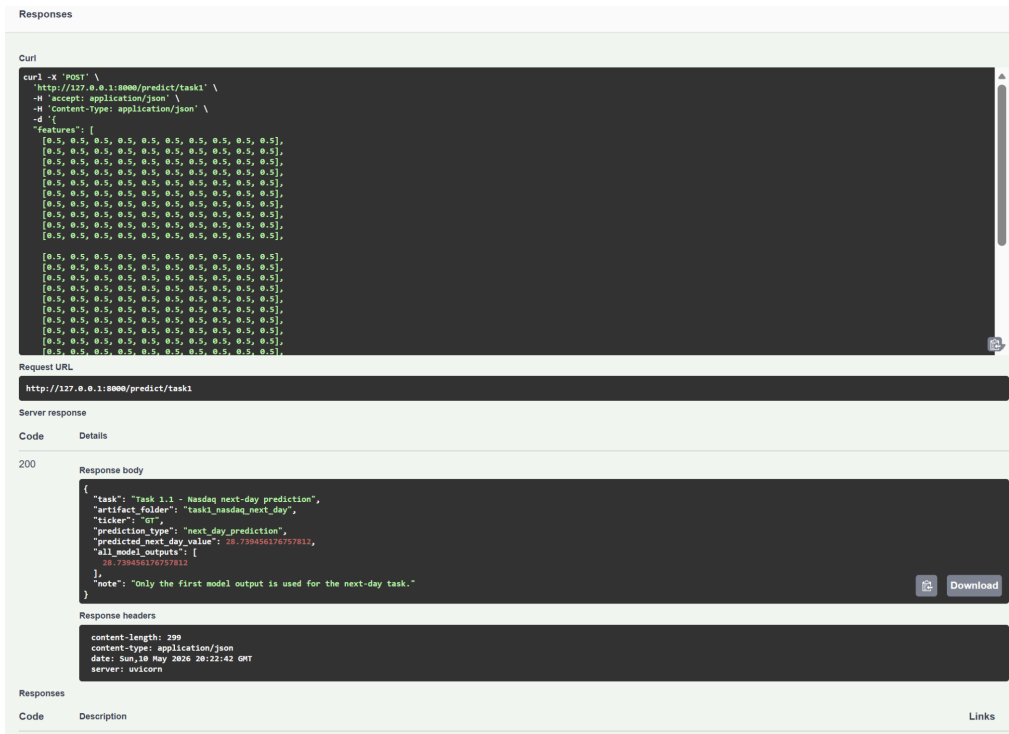
1. POST /predict/task1
2. Click: Try it out



- #### 4. Execute

Example input JSON files:

[illegible]



Note: You can do the same for POST /predict/task2 with this example input:

```

[0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5],
[0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5],
[0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5],
[0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5],
[0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5],
[0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5, 0.5]
],
"scaled": true
}

```

With the expected output in JSON file:

```

{
  "task": "Task 2.1 - Vietnam next-day prediction",
  "artifact_folder": "task2_vietnam_next_day",
  "ticker": "VCB",
  "prediction_type": "next_day_prediction",
  "predicted_next_day_value": 35521.796875,
  "all_model_outputs": [
    35521.796875
  ],
  "note": "Only the first model output is used for the next-day task."
}

```

7.2 Model as SaaS

I built a Streamlit web interface that acts as a lightweight financial tool. Users can select a deployed model, enter recent stock prices through an editable table or CSV upload, and receive a predicted next-day price. The interface calls the FastAPI backend and displays the result using clear metrics, charts, and tables.

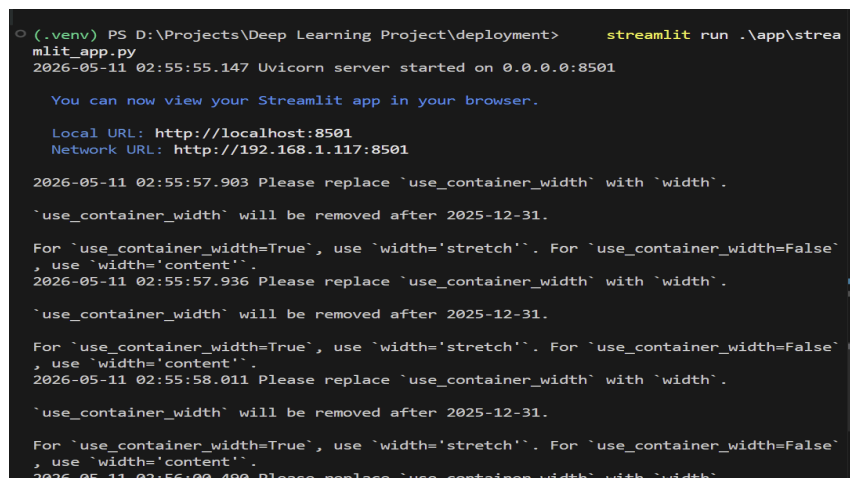
7.2.1 How to start the web interface

Start the Streamlit interface:

```

cd "D:\Projects\Deep Learning Project\deployment"
streamlit run .\app\streamlit_app.py

```



```

(.venv) PS D:\Projects\Deep Learning Project\deployment> streamlit run .\app\streamlit_app.py
2026-05-11 02:55:55.147 Uvicorn server started on 0.0.0.0:8501

You can now view your Streamlit app in your browser.

Local URL: http://localhost:8501
Network URL: http://192.168.1.117:8501

2026-05-11 02:55:57.903 Please replace `use_container_width` with `width`.
`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
2026-05-11 02:55:57.936 Please replace `use_container_width` with `width`.
`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
2026-05-11 02:55:58.011 Please replace `use_container_width` with `width`.
`use_container_width` will be removed after 2025-12-31.

For `use_container_width=True`, use `width='stretch'`. For `use_container_width=False`, use `width='content'`.
2026-05-11 02:56:00.490 Please replace `use_container_width` with `width`.

```

Figure 7.3: Streamlit web interface running locally

The web interface runs at: <http://localhost:8501/>

7.2.1 How the interface calls the API

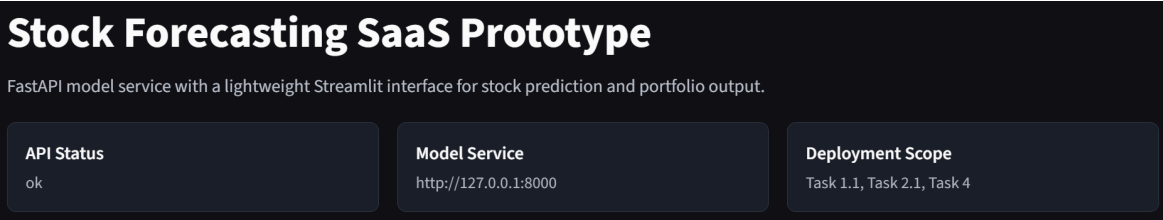


Figure 7.4: Streamlit API Status

The Streamlit interface calls the FastAPI backend at <http://127.0.0.1:8000>. It first checks the backend status using /health and loads available models using /models. The status card displays "ok" when the API is reachable.

Web App Action	API Endpoint
Load API status	GET /health
Load model metadata	GET /models
Predict Task 1.1	POST /predict/task1
Predict Task 2.1	POST /predict/task2
Load Task 4 portfolio	GET /portfolio/task4

Figure 7.5: Mapping table between user action and API’s Endpoint

7.2.3 How to input values and read predictions.

Proceed following steps:

In Streamlit:

- 1. Go to the Price Prediction tab.
- 2. Select: “Task 1.1 - Nasdaq next-day prediction” or other modes.

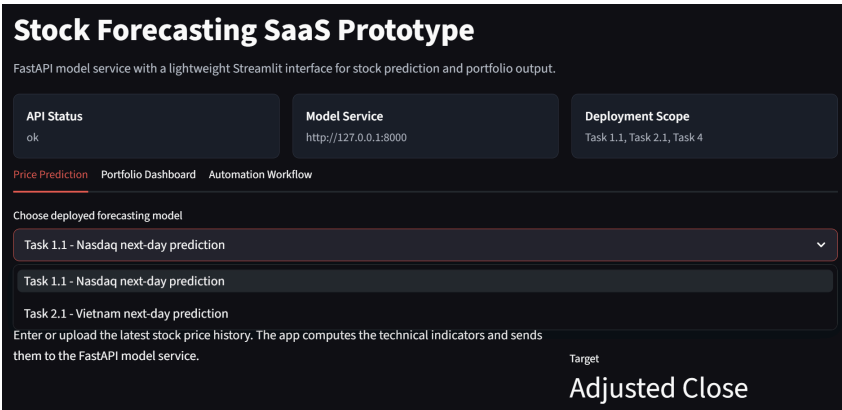


Figure 7.6: What it looks like at this step

3. Choose input mode: **Edit recent price table** (this is more user-friendly, rather than pasting a JSON file or uploading a CSV).

averages, and volatility are computed automatically.

> Model feature order

> Model metadata

Input method

☒ Edit recent price table ☐ Upload CSV ☐ Advanced JSON

Recommended for demos: edit recent stock prices directly. Technical indicators are computed automatically from this table.

Date	Open	High	Low	Close	Adjusted Close	Volume
2026-03-31	28.028	28.3643	27.664	28	27.944	2500000
2026-04-01	28.2055	28.544	27.8418	28.1799	28.1236	2539734
2026-04-02	28.3657	28.7061	28.0076	28.3478	28.2911	2577884
2026-04-03	28.4983	28.8403	28.1508	28.4927	28.4357	2612928
2026-04-06	28.5957	28.9498	28.2526	28.6066	28.5494	2643471
2026-04-07	28.6542	29.0287	28.3104	28.6845	28.6271	2668294
2026-04-08	28.6742	29.0702	28.3301	28.7255	28.6681	2686408
2026-04-09	28.6601	29.0778	28.3162	28.733	28.6755	2697090
2026-04-10	28.6204	29.0582	28.2769	28.7136	28.6562	2699915
2026-04-13	28.566	29.0215	28.2232	28.6774	28.6201	2694770

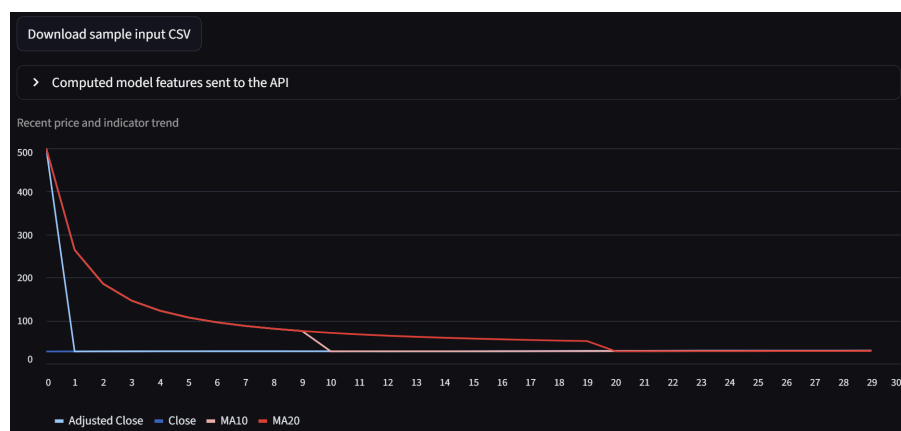
Download sample input CSV

4. Double-click on Adjusted Close and edit manually.

Recommended for demos: edit recent stock prices directly. Technical indicators are computed automatically from this table.

Date	Open	High	Low	Close	Adjusted Close	Volume
2026-03-31	28.028	28.3643	27.664	28	500	2500000
2026-04-01	28.2055	28.544	27.8418	28.1799	28.1236	2539734
2026-04-02	28.3657	28.7061	28.0076	28.3478	28.2911	2577884
2026-04-03	28.4983	28.8403	28.1508	28.4927	28.4357	2612928
2026-04-06	28.5957	28.9498	28.2526	28.6066	28.5494	2643471
2026-04-07	28.6542	29.0287	28.3104	28.6845	28.6271	2668294
2026-04-08	28.6742	29.0702	28.3301	28.7255	28.6681	2686408
2026-04-09	28.6601	29.0778	28.3162	28.733	28.6755	2697090
2026-04-10	28.6204	29.0582	28.2769	28.7136	28.6562	2699915
2026-04-13	28.566	29.0215	28.2232	28.6774	28.6201	2694770

5. Visualize the recent trends



6. Click on “Predict Next-Day price” (also validate the API response)

Predict Next-Day Price

Prediction completed.

Predicted Next-Day Price

28.08

Full API response

```
{
  "task": "Task 1.1 - Nasdaq next-day prediction",
  "artifact_folder": "task1_nasdaq_next_day",
  "ticker": "GT",
  "prediction_type": "next_day_prediction",
  "predicted_next_day_value": 28.082019805908203,
  "all_model_outputs": [
    0 : 28.082019805908203
  ]
  "note": "Only the first model output is used for the next-day task."
}
```

Note: We can do the same procedure for the Task 2 model. Now we proceed to Task 4.

Proceed following steps:

In Streamlit:

1. Go to “Portfolio Dashboard”
2. Click Portfolio Recommendation

Stock Forecasting SaaS Prototype

FastAPI model service with a lightweight Streamlit interface for stock prediction and portfolio output.

API Status
ok

Model Service
http://127.0.0.1:8000

Deployment Scope
Task 1.1, Task 2.1, Task 4

Price Prediction **Portfolio Dashboard** Automation Workflow

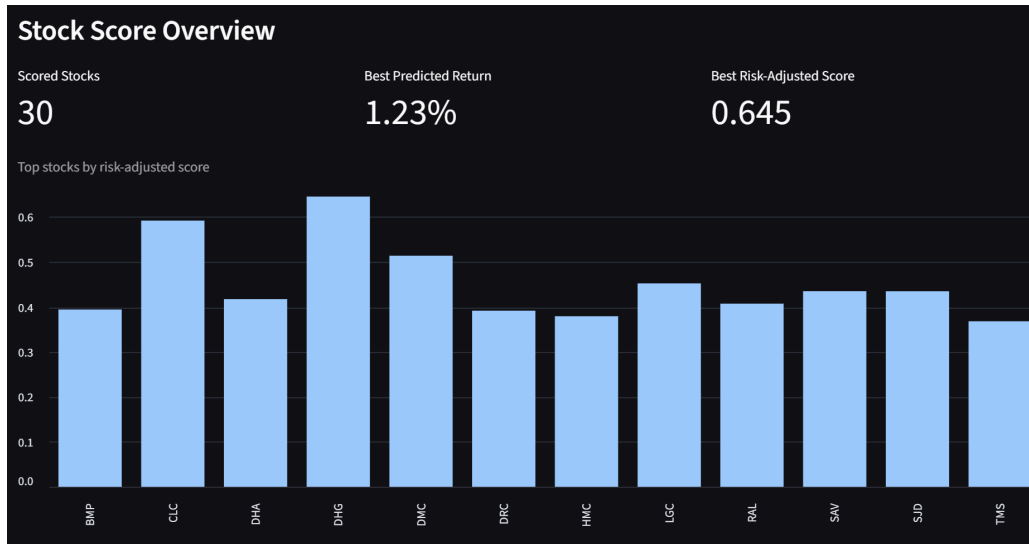
Vietnam Portfolio Recommendation

Task 4 returns the saved portfolio output generated from the portfolio scoring workflow. The output includes prudent, risk-taking, and stock-score tables.

> Model metadata

Load Portfolio Recommendation

3. Stock Score Overview



4. Prudent Portfolio

Prudent Portfolio

Holdings: 8
Total Weight: 100.00%
Avg Predicted Return: 0.63%

Ticker	Trailing Return 20D	Trailing Return 60D	Volatility 20D	Volatility 60D	Downside Volatility 60D	Max Drawdown 120D	Sharpe 60D	Nonzero Volume Rate
SJD	-0.0323	0.0033	0.0065	0.0087	0.0045	-0.2486	0.1693	1
DMC	-0.0217	0.0714	0.0056	0.0112	0.0074	-0.1844	1.7143	0.9917
DHG	0.0201	0.1602	0.0107	0.0126	0.0048	-0.1315	3.227	1
TMS	-0.0202	-0.0838	0.0131	0.0157	0.0109	-0.1942	-1.3541	0.9917
CLC	0.0904	0.1036	0.015	0.016	0.0129	-0.1622	1.7543	0.9583
SAV	-0.043	0.156	0.0163	0.017	0.0107	-0.3404	2.3869	0.9833
RAL	-0.0825	0.1056	0.0151	0.019	0.008	-0.3083	1.5448	1
BMP	-0.08	0.0873	0.021	0.0222	0.0128	-0.2071	1.1718	1

5. We can view other dashboards like:

- Portfolio Weights, Return-Ranking Metrics.
- Risk-Taking Portfolio depends on your references.

7.3 AI Engineering Workflow

The goal of the AI engineering workflow is to automate the stock prediction lifecycle from data collection to transformation, prediction, storage, and visualization. The workflow is designed for the deployed Task 1.1, Task 2.1, and Task 4 models.

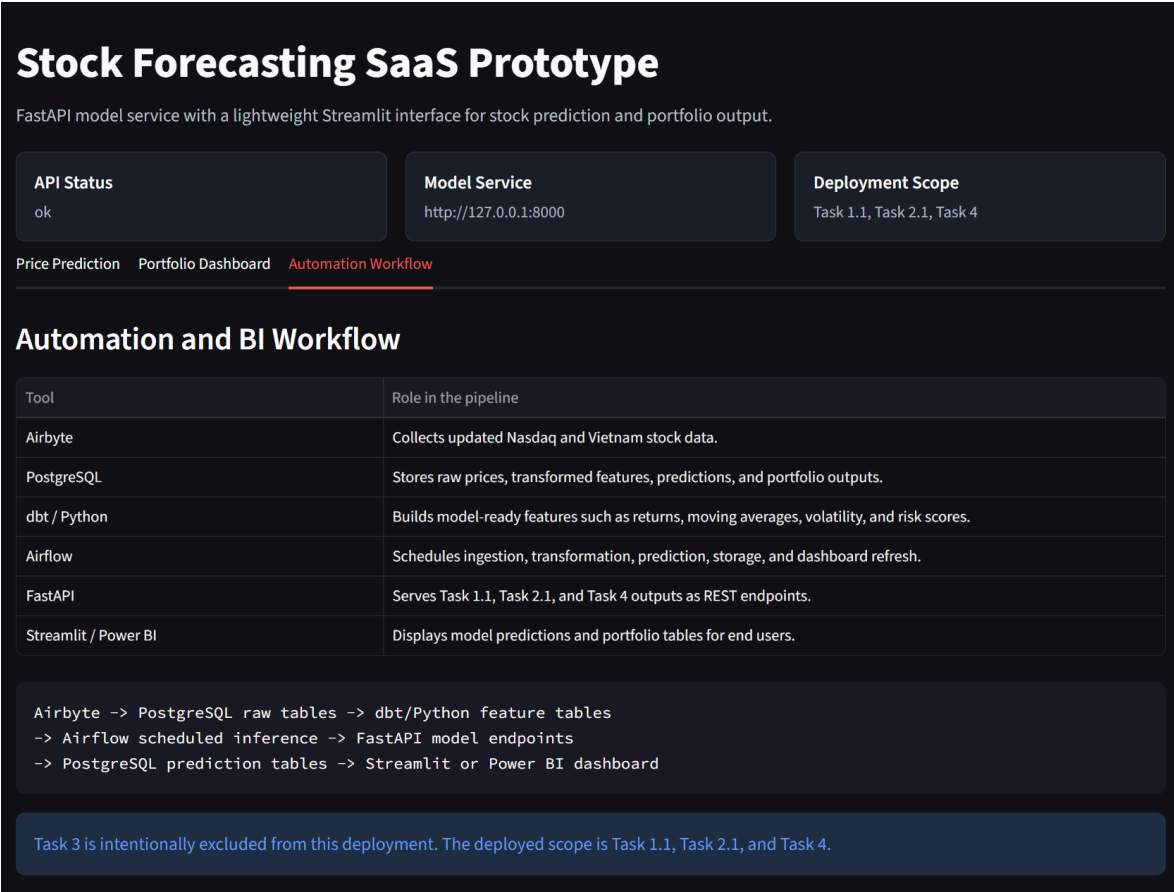


Figure 7.7: Proposed AI engineering workflow

7.3.1 How each tool is used:

Tool	How It Is Used
Airbyte	Automatically pulls updated Nasdaq and Vietnam stock data
PostgreSQL	Stores raw stock data, feature tables, prediction results, and portfolio recommendations
dbt / Python	Cleans raw data and computes model features such as moving averages, log returns, and volatility
Airflow	Schedules the pipeline to run regularly and coordinates all tasks
FastAPI	Loads trained models and exposes prediction endpoints
Streamlit	Displays predictions, portfolio tables, and charts to users
Power BI / Tableau / Superset	Optional BI dashboard tools using exported CSV files

The local prototype implements FastAPI and Streamlit. PostgreSQL, Airbyte, dbt, and Airflow are included in the production workflow design to show how the system could be automated in an industry setting.

7.3.2 Step-by-step of how data moves in the pipeline:

Proceed following the steps that are numbered prior:

1. **Airbyte** ingests updated stock price data from Nasdaq and Vietnam market sources.
2. **PostgreSQL** stores the raw data in the raw_stock_prices table.
3. **dbt or Python** validates and transforms the raw data into model-ready feature tables.
4. **Feature engineering** computes log returns, moving averages, volatility, and risk indicators.
5. **Airflow** schedules and monitors the full pipeline, then triggers model prediction jobs or calls the FastAPI endpoints.
6. **FastAPI** loads the saved Keras models, scalers, and metadata, then returns predictions.
7. **Prediction results** are **written back to PostgreSQL** in model_predictions and portfolio_recommendations.
8. **Streamlit** reads results through the API and displays predicted prices, portfolio tables, charts, and CSV downloads.
8. Optional BI tools such as Power BI, Tableau, or Superset can visualize the exported prediction tables.

7.3.3 (Optional) Supporting Files for this pipeline

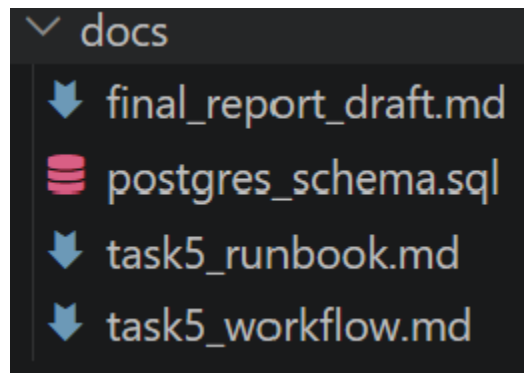


Figure 7.8: Suggested files for this pipeline

The workflow is documented in task5_workflow.md. The PostgreSQL schema in postgres_schema.sql defines tables for raw prices, model features, predictions, and portfolio recommendations. The runbook provides reproducible commands for starting and testing the system.